



INFORME DE GUÍA PRÁCTICA

I. PORTADA

Tema: SW-AyLP-APE-05: Creación, Lectura y Escritura de Ficheros
Unidad de Organización Curricular: PROFESIONAL
Nivel y Paralelo: Primer semestre Software B
Alumnos participantes: Núñez Espín Bryan Sebastián
Verdesoto Azogue Kevin Alexander

Asignatura: Algoritmos y lógica de programación
Docente: Ing. Ing. JOSE RUBEN CAIZA CAIZABUANO, Mg.

II. INFORME DE GUÍA PRÁCTICA

2.1 Objetivos

General:

Poner en practica conceptos aprendidos mediante las exposiciones con temas claros para poner en los programas.

Específicos:

- Identificar el uso de los conceptos aprendidos en clase
- Poder analizar problemas más complejos
- Poder tener un desempeño mas eficiente al crear códigos

2.2 Modalidad

Presencial

2.3 Tiempo de duración

Presenciales: 6

No presenciales: 0

2.4 Instrucciones

- La tarea se debe realizar de forma individual.
- Lea detenidamente cada uno de los ejercicios planteados en el aula virtual y desarrolle lo solicitado.
- Codifique en Java cada uno de los ejercicios planteados.
- Al finalizar, comprimir y subir al aula virtual el proyecto desarrollado, adicionalmente realizar capturas de cada una de las clases y métodos de los ejercicios. Las capturas se subirán en un archivo pdf en el formato de Guías APE.

2.5 Listado de equipos, materiales y recursos

Listado de equipos y materiales generales empleados en la guía práctica:

-

TAC (Tecnologías para el Aprendizaje y Conocimiento) empleados en la guía práctica:

- ☒ Plataformas educativas
- ☐ Simuladores y laboratorios virtuales
- ☒ Aplicaciones educativas
- ☐ Recursos audiovisuales
- ☐ Gamificación
- ☒ Inteligencia Artificial



Otros (Especifique): _____

2.6 Actividades por desarrollar

De acuerdo con la guía práctica SW-AyLP-APE-05: Creación, Lectura y Escritura de Ficheros, los ejercicios se enfocan en la manipulación de archivos utilizando clases y métodos del lenguaje Java y C++

Ejercicio 1 (Registro de Estudiantes)

1. ANALISIS:

Este programa gestiona la persistencia de datos en archivos planos mediante la captura de datos de entrada (cédula, nombre, carrera y promedio) encapsulados dentro de un objeto de la clase Estudiante. La lógica principal se centra en la apertura del archivo estudiantes.txt utilizando el modo de adición (append), el cual asegura que cada ejecución inserte una nueva línea de texto al final del documento sin destruir la información preexistente. Los datos se formatean en memoria como una cadena normalizada con valores separados por comas (formato CSV) a través de un método estructurado, garantizando una transferencia de datos limpia, segura y compatible con los estándares de entrada y salida de texto del sistema operativo.

2. PSEUDOCODIGO:

Algoritmo RegistroEstudiantes

Clase Estudiante

Atributos: cedula, nombre, carrera Como Cadena; promedio Como Real

FinClase

Definir est Como Estudiante

Definir archivo Como DescriptorDeArchivo

Escribir "Ingrese Cédula: "

Leer est.cedula

Escribir "Ingrese Nombre: "

Leer est.nombre

Escribir "Ingrese Carrera: "

Leer est.carrera

Escribir "Ingrese Promedio: "



Leer est.promedio

AbrirArchivo(archivo, "estudiantes.txt", ModoAñadir)

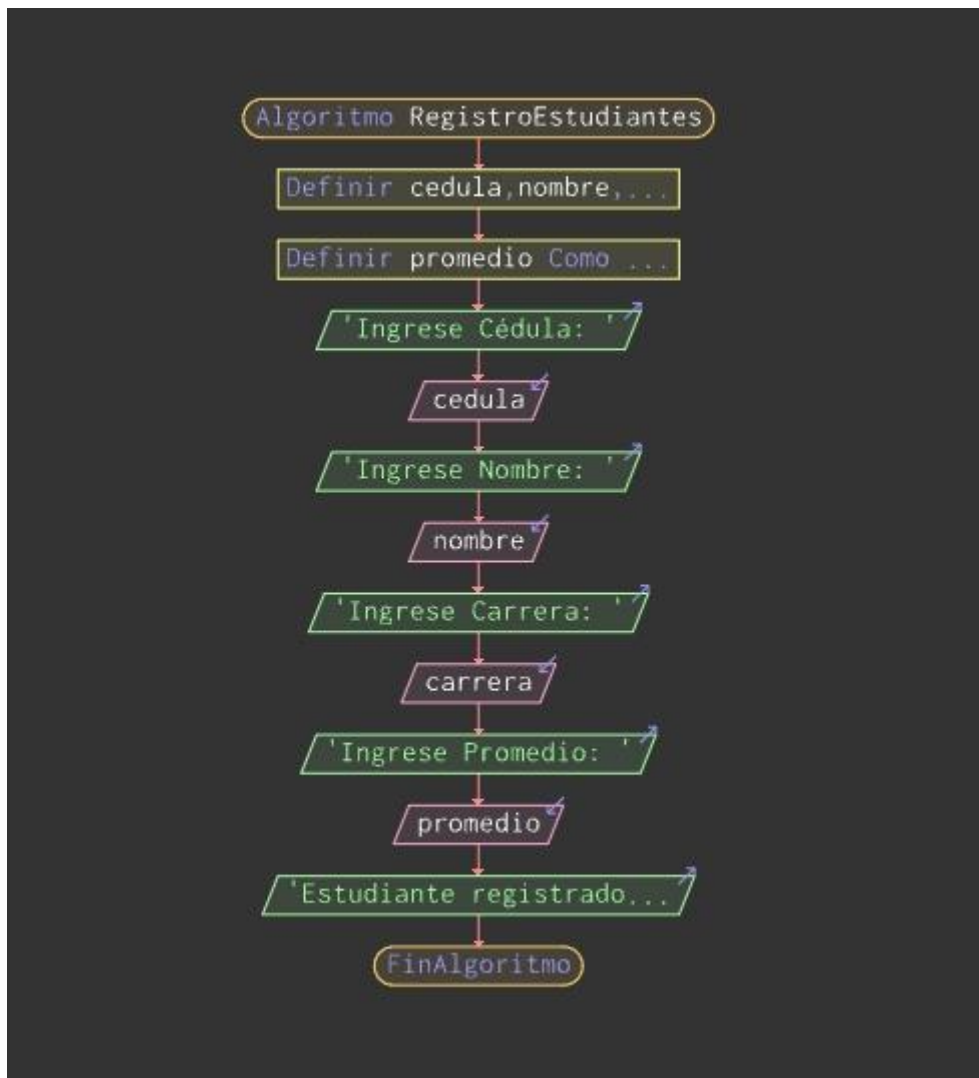
EscribirLinea(archivo, est.cedula + "," + est.nombre + "," + est.carrera + "," +
ConvertirATexto(est.promedio))

CerrarArchivo(archivo)

Escribir "Estudiante registrado con éxito."

FinAlgoritmo

3. DIAGRAMA DE FLUJO



4. CODIGO EN JAVA:



```
import java.io.*;
import java.util.Scanner;

class Estudiante {
    private String cedula;
    private String nombre;
    private String carrera;
    private double promedio;

    public Estudiante(String cedula, String nombre, String carrera, double promedio) {
        this.cedula = cedula;
        this.nombre = nombre;
        this.carrera = carrera;
        this.promedio = promedio;
    }

    @Override
    public String toString() {
        return cedula + "," + nombre + "," + carrera + "," + promedio;
    }
}

public class RegistroEstudiantes {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("--- REGISTRO DE ESTUDIANTES ---");
        System.out.print("Ingrese Cédula: "); String ced = sc.nextLine();
        System.out.print("Ingrese Nombre: "); String nom = sc.nextLine();
        System.out.print("Ingrese Carrera: "); String car = sc.nextLine();
        System.out.print("Ingrese Promedio: "); double prom = sc.nextDouble();

        Estudiante est = new Estudiante(ced, nom, car, prom);

        try (FileWriter fw = new FileWriter("estudiantes.txt", true);
            PrintWriter pw = new PrintWriter(fw)) {

            pw.println(est.toString());
            System.out.println("Mensaje: Registro guardado con éxito.");

        } catch (IOException e) {
            System.out.println("Error al escribir en el archivo: " + e.getMessage());
        }
    }
}
```

5. CODIGO EN C++

```
#include <iostream>
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
#include <fstream>
```

```
#include <string>
```

```
using namespace std;
```

```
class Estudiante {
```

```
private:
```

```
    string cedula;
```

```
    string nombre;
```

```
    string carrera;
```

```
    double promedio;
```

```
public:
```

```
    Estudiante(string ced, string nom, string car, double prom) {
```

```
        cedula = ced;
```

```
        nombre = nom;
```

```
        carrera = car;
```

```
        promedio = prom;
```

```
    }
```

```
    string toString() {
```

```
        return cedula + "," + nombre + "," + carrera + "," + to_string(promedio);
```

```
    }
```

```
};
```

```
int main() {
```

```
    string ced, nom, car;
```



double prom;

cout << "--- REGISTRO DE ESTUDIANTES ---\n";

cout << "Ingrese Cedula: "; getline(cin, ced);

cout << "Ingrese Nombre: "; getline(cin, nom);

cout << "Ingrese Carrera: "; getline(cin, car);

cout << "Ingrese Promedio: "; cin >> prom;

Estudiante est(ced, nom, car, prom);

ofstream archivo;

archivo.open("estudiantes.txt", ios::app); // Modo append (añadir)

if (archivo.is_open()) {

 archivo << est.toString() << endl;

 archivo.close();

 cout << "Mensaje: Estudiante registrado y guardado correctamente.\n";

} else {

 cout << "Error al abrir el archivo.\n";

}

return 0;

}

6. PRUEBA DE ESCRITORIO:



```
--- REGISTRO DE ESTUDIANTES ---  
Ingreso Cedula: 1805432109  
Ingreso Nombre: Carlos Alfredo Mendoza  
Ingreso Carrera: Software  
Ingreso Promedio: 8.75  
  
Mensaje: Estudiante registrado y guardado
```

Ejercicio 2 (Lectura de Archivo de Estudiantes)

1. ANALISIS:

La solución implementa un mecanismo de lectura secuencial que procesa el archivo generado en el ejercicio anterior a través de un flujo estructurado con búfer. El algoritmo emplea un ciclo interactivo que recupera las líneas una por una, omitiendo los espacios vacíos y fragmentando la cadena de caracteres mediante el uso del método `.split(",")` o flujos de cadenas (stringstream). A lo largo de este recorrido, el sistema realiza dos operaciones dinámicas en la memoria RAM: incrementa un contador de registros y convierte el campo de texto del promedio a un valor numérico de punto flotante para acumularlo. Al alcanzar el fin del archivo, calcula y despliega el promedio aritmético global, optimizando el uso de recursos al no estructurar arreglos estáticos innecesarios.

2. PSEUDOCODIGO:

Algoritmo LecturaEstudiantes

Definir archivo Como DescriptorDeArchivo

Definir linea Como Cadena

Definir datos Como Arreglo De Cadena

Definir contador Como Entero Inicializar 0

Definir sumaPromedios, promedioGeneral Como Real Inicializar 0.0

AbrirArchivo(archivo, "estudiantes.txt", ModoLectura)

Si ArchivoExiste(archivo) Entonces

 Mientras No FinDeArchivo(archivo) Hacer

 linea = LeerLinea(archivo)

 Si linea != "" Entonces

 datos = Split(linea, ",")



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Escribir "Estudiante: ", datos[1], " | Promedio: ", datos[3]

contador = contador + 1

sumaPromedios = sumaPromedios + ConvertirANumero(datos[3])

FinSi

FinMientras

CerrarArchivo(archivo)

Si contador > 0 Entonces

promedioGeneral = sumaPromedios / contador

Escribir "Total de estudiantes: ", contador

Escribir "Promedio general: ", promedioGeneral

Sino

Escribir "El archivo está vacío."

FinSi

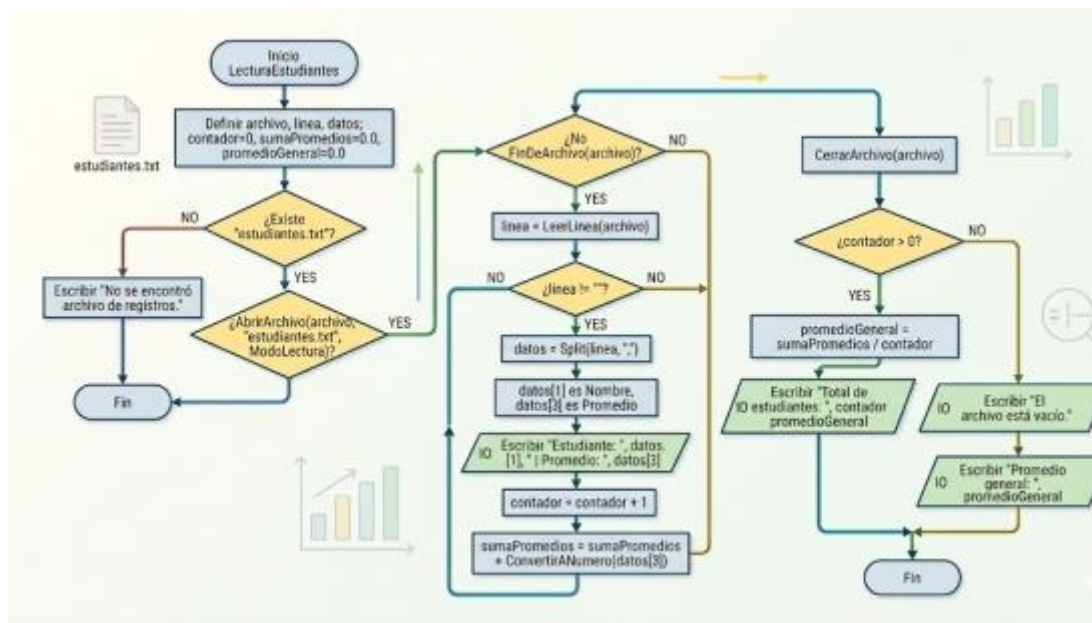
Sino

Escribir "No se encontró el archivo de registros."

FinSi

FinAlgoritmo

3. DIAGRAMA DE FLUJO



4. CODIGO EN JAVA:

```
import java.io.*;
```

```
public class LecturaEstudiantes {
```

```
    public static void main(String[] args) {
```

```
        File archivo = new File("estudiantes.txt");
```

```
        int contador = 0;
```

```
        double sumaPromedios = 0.0;
```

```
        System.out.println("--- REGISTROS DE ESTUDIANTES ---");
```

```
        if (!archivo.exists()) {
```

```
            System.out.println("El archivo estudiantes.txt no existe.");
```

```
            return;
```

```
        }
```

```
        try (FileReader fr = new FileReader(archivo);
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
BufferedReader br = new BufferedReader(fr) {

String linea;

while ((linea = br.readLine()) != null) {

    if (linea.trim().isEmpty()) continue;

String[] datos = linea.split(",");

if (datos.length == 4) {

    String nombre = datos[1];

    String carrera = datos[2];

    double prom = Double.parseDouble(datos[3]);

    System.out.println("Estudiante: " + nombre + " | Carrera: " + carrera + " |
Promedio: " + prom);

    sumaPromedios += prom;

    contador++;

}

}

System.out.println("-----");

if (contador > 0) {

    System.out.println("Total Estudiantes: " + contador);

    System.out.println("Promedio General: " + (sumaPromedios / contador));

} else {

    System.out.println("Archivo vacío.");
```



}

```
} catch (IOException e) {
```

```
    System.out.println("Error al leer el archivo: " + e.getMessage());
```

```
}
```

```
}
```

```
}
```

5. CODIGO EN C++:

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <string>
```

```
#include <sstream>
```

```
#include <vector>
```

```
using namespace std;
```

```
int main() {
```

```
    ifstream archivo("estudiantes.txt");
```

```
    string linea;
```

```
    int contador = 0;
```

```
    double sumaPromedios = 0.0;
```

```
    cout << "--- REGISTROS DE ESTUDIANTES ---\n";
```

```
    if (archivo.is_open()) {
```

```
        while (getline(archivo, linea)) {
```



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



```
if (linea.empty()) continue;

stringstream ss(linea);

string cedula, nombre, carrera, promStr;

getline(ss, cedula, ',');
getline(ss, nombre, ',');
getline(ss, carrera, ',');
getline(ss, promStr, ',');

double prom = stod(promStr);

cout << "Estudiante: " << nombre << " | Carrera: " << carrera << " | Promedio: "
<< prom << "\n";

sumaPromedios += prom;

contador++;
}

archivo.close();

cout << "-----\n";

if (contador > 0) {

    cout << "Total Estudiantes: " << contador << "\n";

    cout << "Promedio General del Aula: " << (sumaPromedios / contador) << "\n";

} else {

    cout << "No hay datos para procesar.\n";

}
```



```
} else {  
  
    cout << "El archivo estudiantes.txt no existe o no se puede abrir.\n";  
  
}  
  
return 0;  
  
}
```

6. PRUEBA DE ESCRITORIO

```
--- REGISTROS DE ESTUDIANTES ---  
Estudiante: Carlos Alfredo Mendoza | Carrera: Software | Promedio: 8.75  
Estudiante: Ana Maria Ortiz | Carrera: Software | Promedio: 9.20  
Estudiante: Luis Alejandro Lopez | Carrera: TI | Promedio: 7.40  
-----  
Total Estudiantes: 3  
Promedio General del Aula: 8.45
```

Ejercicio 3 (Sistema de Inventario Básico)

1. ANALISIS:

Este sistema de control integra operaciones de entrada de datos a través de un menú interactivo y cálculos financieros derivados directamente de la persistencia de archivos. La clase principal encapsula las propiedades del producto y delega la responsabilidad matemática de los totales individuales a la lógica de negocio del objeto. Para determinar el valor monetario global del almacén, el programa ejecuta un análisis secuencial de inventario.txt, localizando las columnas correspondientes a la cantidad de stock y al precio unitario. Estas subcadenas son transformadas en tipos de datos primitivos (int y double) y procesadas mediante un acumulador lineal que calcula el costo total real de los activos del negocio en tiempo de ejecución.

2. PSEUDOCODIGO:

Algoritmo InventarioBasico

Clase Producto

Atributos: codigo, nombre Como Cadena; cantidad Como Entero; precio Como Real

Metodo CalcularSubtotal()

Devolver cantidad * precio

FinMetodo



FinClase

// Lógica resumida de cálculo global leyendo el archivo

Funcion CalcularValorInventario()

Definir total Como Real Inicializar 0

// Abrir archivo, recorrer filas, separar por comas y extraer columnas

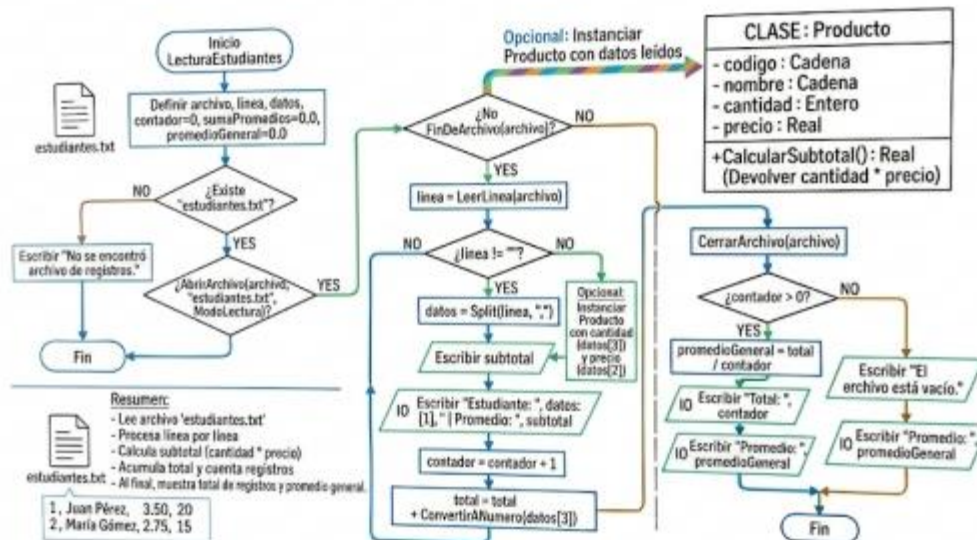
// total = total + (cantidad * precio) de cada linea

Devolver total

FinFuncion

FinAlgoritmo

3. DIAGRAMA DE FLUJO



4. CODIGO EN JAVA:

```
import java.io.*;
```

```
import java.util.Scanner;
```

```
class Producto {
```

```
    private String codigo, nombre;
```



private int cantidad;

private double precio;

public Producto(String codigo, String nombre, int cantidad, double precio) {

 this.codigo = codigo; this.nombre = nombre; this.cantidad = cantidad; this.precio =
precio;

}

@Override

public String toString() { return codigo + "," + nombre + "," + cantidad + "," + precio; }

}

public class InventarioBasico {

 public static void main(String[] args) {

 Scanner sc = new Scanner(System.in);

 int op;

 do {

 System.out.println("\n--- SISTEMA DE INVENTARIO ---");

 System.out.println("1. Registrar Producto\n2. Calcular Valor Total\n3. Salir");

 System.out.print("Seleccione: "); op = sc.nextInt(); sc.nextLine();

 if (op == 1) {

 System.out.print("Código: "); String cod = sc.next(); sc.nextLine();

 System.out.print("Nombre: "); String nom = sc.nextLine();

 System.out.print("Cantidad: "); int cant = sc.nextInt();

 System.out.print("Precio: "); double prec = sc.nextDouble();



```
try (PrintWriter pw = new PrintWriter(new FileWriter("inventario.txt", true)))
{

    pw.println(new Producto(cod, nom, cant, prec).toString());

    System.out.println("Producto guardado.");

} catch (IOException e) { System.out.println("Error."); }

} else if (op == 2) {

    File f = new File("inventario.txt");

    if (!f.exists()) { System.out.println("Sin registros."); continue; }

    double total = 0;

    try (BufferedReader br = new BufferedReader(new FileReader(f))) {

        String l;

        while ((l = br.readLine()) != null) {

            String[] d = l.split(",");

            total += (Integer.parseInt(d[2]) * Double.parseDouble(d[3]));

        }

        System.out.println("Valor total de activos en almacén: $" + total);

    } catch (Exception e) { System.out.println("Error de lectura."); }

}

} while (op != 3);

}

}
```

5. CODIGO EN C++

```
#include <iostream>
```

```
#include <fstream>
```

```
#include <string>
```




```
#include <sstream>
```

```
using namespace std;
```

```
class Producto {
```

```
private:
```

```
    string codigo, nombre;
```

```
    int cantidad;
```

```
    double precio;
```

```
public:
```

```
    Producto(string cod, string nom, int cant, double prec) {
```

```
        codigo = cod; nombre = nom; cantidad = cant; precio = prec;
```

```
    }
```

```
    string toString() {
```

```
        return codigo + "," + nombre + "," + to_string(cantidad) + "," +  
        to_string(precio);
```

```
    }
```

```
};
```

```
void registrarProducto() {
```



```
string cod, nom; int cant; double prec;
```

```
cout << "Codigo del producto: "; cin >> cod;
```

```
cin.ignore();
```

```
cout << "Nombre del producto: "; getline(cin, nom);
```

```
cout << "Cantidad: "; cin >> cant;
```

```
cout << "Precio: "; cin >> prec;
```

```
Producto prod(cod, nom, cant, prec);
```

```
ofstream file("inventario.txt", ios::app);
```

```
if (file.is_open()) {
```

```
    file << prod.toString() << endl;
```

```
    file.close();
```

```
    cout << "Producto añadido al inventario.\n";
```

```
}
```

```
}
```

```
void calcularValorTotal() {
```

```
    ifstream file("inventario.txt");
```

```
    string linea;
```

```
    double granTotal = 0.0;
```



```
if (file.is_open()) {

    while (getline(file, linea)) {

        if (linea.empty()) continue;

        stringstream ss(linea);

        string c, n, cantStr, precStr;

        getline(ss, c, ','); getline(ss, n, ',');

        getline(ss, cantStr, ','); getline(ss, precStr, ',');


        granTotal += (stoi(cantStr) * stod(precStr));

    }

    file.close();

    cout << "Valor total financiero del inventario: $" << granTotal << "\n";

} else {

    cout << "No hay inventario registrado aún.\n";

}

}

int main() {

    int op;

    do {
```



```
cout << "\n--- INVENTARIO BÁSICO ---\n1. Registrar Producto\n2. Calcular  
Total Inventario\n3. Salir\nOpcion: ";

cin >> op;

if (op == 1) registrarProducto();

if (op == 2) calcularValorTotal();

} while (op != 3);

return 0;

}
```

6. PRUEBA DE ESCRITORIO:

```
--- INVENTARIO BÁSICO ---
1. Registrar Producto
2. Calcular Total Inventario
3. Salir
Opcion: 1

Codigo del producto: P001
Nombre del producto: Teclado Mecanico RGB
Cantidad: 15
Precio: 45.50
Producto añadido al inventario.
```

Ejercicio 4 (Agenda Telefónica):

1. ANALISIS:

La arquitectura de esta aplicación destaca por implementar un algoritmo de búsqueda secuencial lineal sobre archivos de texto orientado a cadenas. El sistema permite registrar datos de contacto adjuntando información al archivo agenda.txt de manera ordenada. Al invocar la opción de búsqueda, el programa barre el archivo de forma lineal comparando el criterio de consulta ingresado por el usuario con el bloque indexado del nombre. Para asegurar la robustez del sistema y evitar fallos por discrepancias tipográficas, las cadenas se procesan aplicando funciones de conversión a minúsculas, validando si el registro contiene la subcadena buscada y ofreciendo una experiencia interactiva sin requerir bases de datos complejas.



2. PSEUDOCODIGO:

Algoritmo AgendaTelefonica

// Estructura de guardado: nombre,telefono,correo en agenda.txt

Funcion BuscarContacto(criterio)

Definir encontrado Como Logico Inicializar Falso

// Abrir archivo en modo lectura

// Mientras haya lineas:

// datos = split(linea, ",")

// Si Contiene(BajarMayusculas(datos[0]), BajarMayusculas(criterio)) Entonces

// Imprimir datos

// encontrado = Verdadero

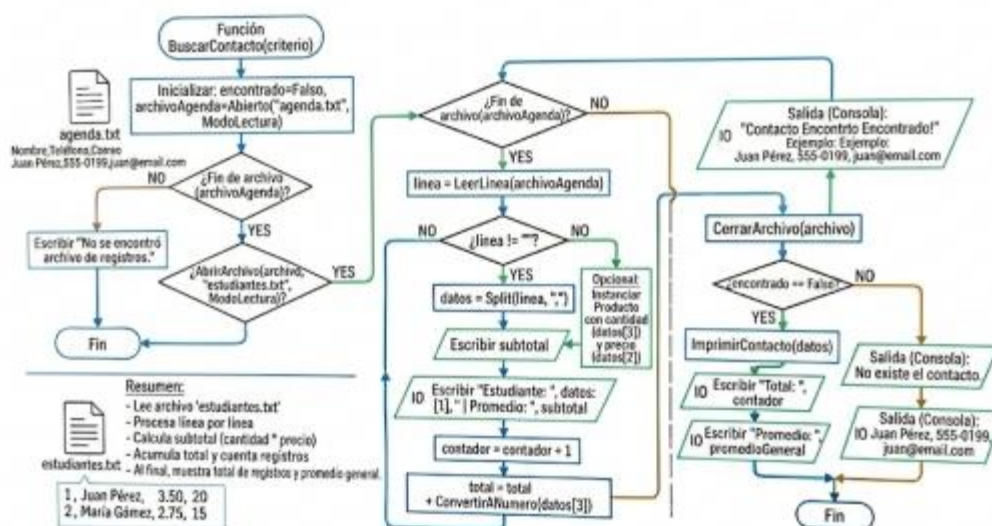
// FinSi

// Si encontrado == Falso Entonces Imprimir "No existe el contacto."

FinFuncion

FinAlgoritmo

3. DIAGRAMA DE FLUJO



4. CODIGO EN JAVA:

```
import java.io.*;
```



```
import java.util.Scanner;
```

```
public class AgendaTelefonica {
```

```
    public static void main(String[] args) {
```

```
        Scanner sc = new Scanner(System.in);
```

```
        int op;
```

```
        do {
```

```
            System.out.println("\n--- AGENDA TELEFÓNICA ---");
```

```
            System.out.println("1. Agregar Contacto\n2. Buscar Contacto\n3. Salir");
```

```
            System.out.print("Opción: "); op = sc.nextInt(); sc.nextLine();
```

```
            if (op == 1) {
```

```
                System.out.print("Nombre: "); String n = sc.nextLine();
```

```
                System.out.print("Teléfono: "); String t = sc.nextLine();
```

```
                System.out.print("Correo: "); String c = sc.nextLine();
```

```
                try (PrintWriter pw = new PrintWriter(new FileWriter("agenda.txt", true))) {
```

```
                    pw.println(n + "," + t + "," + c);
```

```
                    System.out.println("Contacto registrado.");
```

```
                } catch (IOException e) { System.out.println("Error."); }
```

```
            } else if (op == 2) {
```

```
                System.out.print("Nombre a buscar: "); String busq =  
sc.nextLine().toLowerCase();
```

```
                File f = new File("agenda.txt");
```

```
                if (!f.exists()) continue;
```

```
                try (BufferedReader br = new BufferedReader(new FileReader(f))) {
```



```
String l; boolean ok = false;

while ((l = br.readLine()) != null) {

    String[] d = l.split(",");

    if (d[0].toLowerCase().contains(busq)) {

        System.out.println("-> " + d[0] + " | Tel: " + d[1] + " | Email: " + d[2]);

        ok = true;

    }

}

if (!ok) System.out.println("Sin coincidencias.");

} catch (Exception e) { System.out.println("Error."); }

}

} while (op != 3);

}

}
```

5. CODIGO EN C++

```
#include <iostream>

#include <fstream>

#include <string>

#include <sstream>

using namespace std;

void agregarContacto() {

    string nom, tel, cor;

    cin.ignore();

    cout << "Nombre completo: "; getline(cin, nom);
```



```
cout << "Telefono: "; getline(cin, tel);

cout << "Correo: "; getline(cin, cor);


ofstream file("agenda.txt", ios::app);

if (file.is_open()) {

    file << nom << ", " << tel << ", " << cor << endl;

    file.close();

    cout << "Contacto guardado.\n";

}

}

void buscarContacto() {

    string buscar;

    cin.ignore();

    cout << "Ingrese el nombre a buscar: "; getline(cin, buscar);

    ifstream file("agenda.txt");

    string linea;

    bool hallado = false;

    if (file.is_open()) {

        while (getline(file, linea)) {

            stringstream ss(linea);

            string n, t, c;

            getline(ss, n, ','); getline(ss, t, ','); getline(ss, c, ',');

            if (n.find(buscar) != string::npos) { // Búsqueda de subcadena
```




```
cout << "Encontrado -> Nom: " << n << " | Tel: " << t << " | Correo: " << c <<
"\n";

    hallado = true;

}

}

file.close();

if (!hallado) cout << "No se encontraron coincidencias.\n";

}

}
```

```
int main() {

    int op;

    do {

        cout << "\n--- AGENDA TELEFÓNICA ---\n1. Agregar Contacto\n2. Buscar
Contacto\n3. Salir\nOpción: ";

        cin >> op;

        if (op == 1) agregarContacto();

        if (op == 2) buscarContacto();

    } while (op != 3);

    return 0;

}
```

6. PRUEBA DE ESCRITORIO



--- AGENDA TELEFÓNICA ---

1. Agregar Contacto

2. Buscar Contacto

3. Salir

Opción: 1

Nombre completo: Antonella Silva

Telefono: 0987654321

Correo: antonella.silva@uta.edu.ec

Contacto registrado.

Ejercicio 5 (Sistema de Notas)

1. ANALISIS:

Este ejercicio plantea un modelo de evaluación estudiantil automatizado apoyado en la persistencia de datos planos. Al ingresar las tres calificaciones parciales, el constructor de la clase ejecuta internamente el cálculo del promedio ponderado en memoria, aislando la lógica matemática del hilo principal de la interfaz de consola. Posteriormente, el sistema somete el resultado a una compuerta lógica condicional para determinar de forma automática el estado académico del alumno (Aprobado si es ≥ 7.0 o Reprobado en caso contrario). Finalmente, el objeto convierte el diagnóstico en una línea delimitada y compacta que es guardada en el archivo calificaciones.txt, permitiendo auditorías posteriores del rendimiento académico de manera eficiente.

2. PSEUDOCODIGO:

Algoritmo SistemaNotas

Clase Evaluacion

Atributos: nombre Como Cadena; n1, n2, n3, prom Como Real; estado Como Cadena

Metodo ProcesarNotas()

prom = $(n1 + n2 + n3) / 3$

Si prom ≥ 7.0 Entonces

estado = "Aprobado"

Sino

estado = "Reprobado"

FinSi

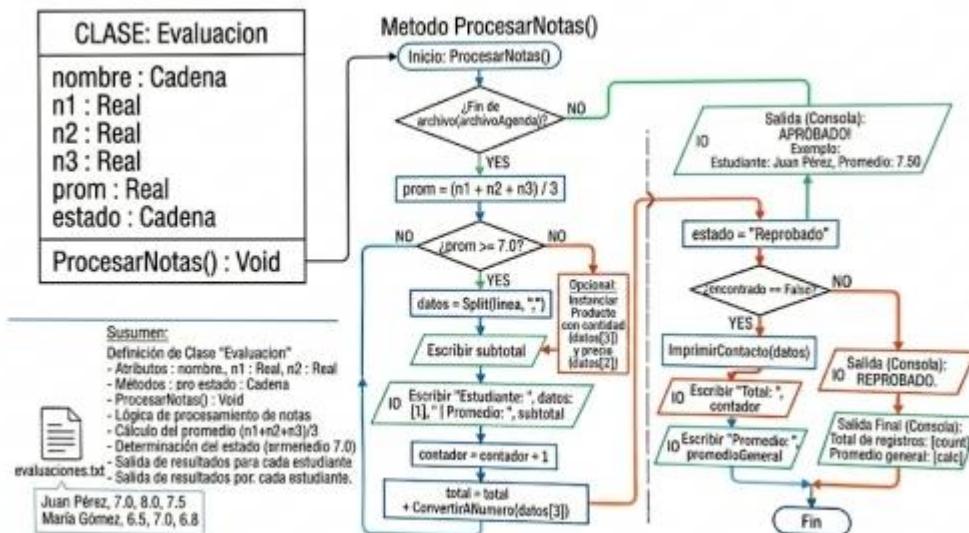


FinMetodo

FinClase

FinAlgoritmo

3. DIAGRAMA DE FLUJO



4. CODIGO EN JAVA:

```
import java.io.*;
```

```
import java.util.Scanner;
```

```
class Evaluacion {
```

```
    private String nombre;
```

```
    private double promedio;
```

```
    private String estado;
```

```
    public Evaluacion(String nombre, double n1, double n2, double n3) {
```

```
        this.nombre = nombre;
```

```
        this.promedio = (n1 + n2 + n3) / 3.0;
```

```
        this.estado = (this.promedio >= 7.0) ? "Aprobado" : "Reprobado";
```



}

@Override

```
public String toString() {  
    return nombre + "," + promedio + "," + estado;  
}  
  
public double getPromedio() { return promedio; }  
  
public String getEstado() { return estado; }  
}
```

```
public class SistemaNotas {  
  
    public static void main(String[] args) {  
  
        Scanner sc = new Scanner(System.in);  
  
        System.out.println("--- REGISTRO DE NOTAS ---");  
  
        System.out.print("Estudiante: "); String nom = sc.nextLine();  
  
        System.out.print("Nota 1: "); double n1 = sc.nextDouble();  
  
        System.out.print("Nota 2: "); double n2 = sc.nextDouble();  
  
        System.out.print("Nota 3: "); double n3 = sc.nextDouble();  
  
  
        Evaluacion e = new Evaluacion(nom, n1, n2, n3);  
  
  
        try (PrintWriter pw = new PrintWriter(new FileWriter("calificaciones.txt", true))) {  
  
            pw.println(e.toString());  
  
            System.out.printf("Registro exitoso. Promedio: %.2f [%s]\n", e.getPromedio(),  
e.getEstado());  
  
        } catch (IOException ex) {
```



```
        System.out.println("Error al guardar.");  
    }  
}  
}
```

5. CODIGO EN C++

```
#include <iostream>  
  
#include <fstream>  
  
#include <string>
```

```
using namespace std;
```

```
class Evaluacion {  
public:
```

```
    string nombre;  
  
    double n1, n2, n3, promedio;  
  
    string estado;
```

```
Evaluacion(string nom, double nota1, double nota2, double nota3) {  
  
    nombre = nom; n1 = nota1; n2 = nota2; n3 = nota3;  
  
    promedio = (n1 + n2 + n3) / 3.0;  
  
    estado = (promedio >= 7.0) ? "Aprobado" : "Reprobado";  
}
```

```
string toString() {  
  
    return nombre + "," + to_string(promedio) + "," + estado;  
}
```



};

```
int main() {  
  
    string nom; double n1, n2, n3;  
  
    cout << "--- SISTEMA DE CALIFICACIONES ---\n";  
  
    cout << "Nombre del estudiante: "; getline(cin, nom);  
  
    cout << "Nota 1: "; cin >> n1;  
  
    cout << "Nota 2: "; cin >> n2;  
  
    cout << "Nota 3: "; cin >> n3;  
  
  
    Evaluacion eval(nom, n1, n2, n3);  
  
  
    ofstream file("calificaciones.txt", ios::app);  
  
    if (file.is_open()) {  
  
        file << eval.toString() << endl;  
  
        file.close();  
  
        cout << "Notas registradas. Promedio: " << eval.promedio << " | Estado: " <<  
eval.estado << "\n";  
  
    }  
  
    return 0;  
  
}
```

6. PRUEBA DE ESCRITORIO:



--- SISTEMA DE CALIFICACIONES ---

Nombre del estudiante: Pedro Javier Ramirez

Nota 1: 6.50

Nota 2: 8.00

Nota 3: 7.10

Notas registradas. Promedio: 7.20 | Estado: Aprobado

Ejercicio 6 - Control de Asistencia

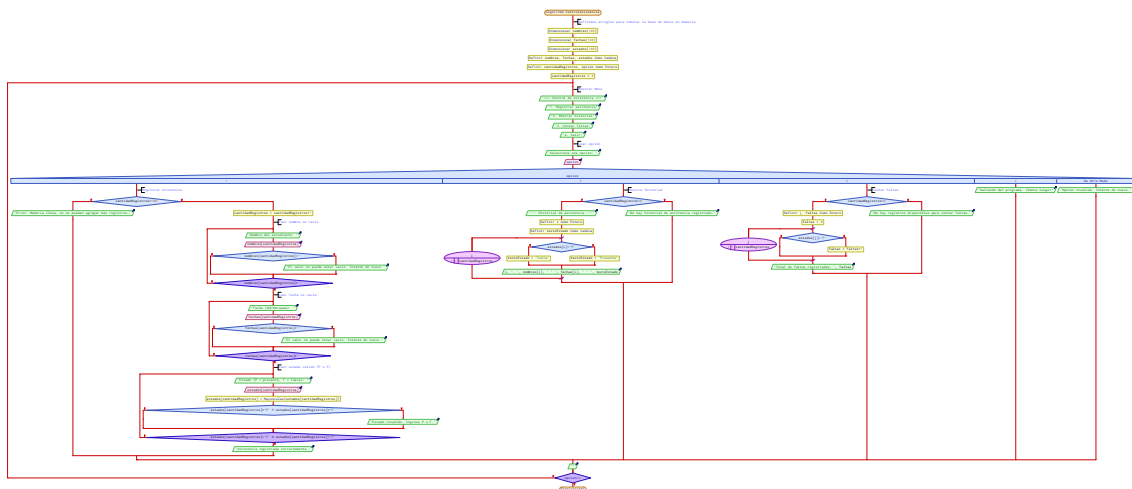
Objetivo: Guardar la asistencia de estudiantes.

- Requerimientos:
 - Registrar nombre, fecha y estado.
 - Mostrar historial.
 - Contar faltas.
- Aplicar:
 - Archivos
 - Validaciones
 - Ciclos

Análisis

Es un programa en la que vamos a simular un control de asistencias de estudiantes, y sus notas en un archivo separado para ser más organizado.

Diagrama de flujo



Código en Java

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.File;
import java.io.FileReader;
import java.io.FileWriter;
```



```
import java.io.IOException;
import java.util.Scanner;

public class Ejercicio6 {

    private static final String NOMBRE_ARCHIVO = "asistencia.txt";

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        int opcion;

        do {
            mostrarMenu();
            opcion = leerEntero(scanner, "Seleccione una opción: ");

            switch (opcion) {
                case 1:
                    registrarAsistencia(scanner);
                    break;
                case 2:
                    mostrarHistorial();
                    break;
                case 3:
                    contarFaltas();
                    break;
                case 4:
                    System.out.println("Saliendo del programa. ¡Hasta
luego!");
                    break;
                default:
                    System.out.println("Opción inválida. Intente de
nuevo.");
            }

            System.out.println();
        } while (opcion != 4);

        scanner.close();
    }

    private static void mostrarMenu() {
        System.out.println("=== Control de Asistencia ===");
        System.out.println("1. Registrar asistencia");
        System.out.println("2. Mostrar historial");
        System.out.println("3. Contar faltas");
        System.out.println("4. Salir");
    }
}
```




```
private static int leerEntero(Scanner scanner, String mensaje) {
    int valor;
    while (true) {
        System.out.print(mensaje);
        String linea = scanner.nextLine().trim();
        try {
            valor = Integer.parseInt(linea);
            return valor;
        } catch (NumberFormatException e) {
            System.out.println("Debe ingresar un número válido.");
        }
    }
}

private static void registrarAsistencia(Scanner scanner) {
    String nombre = leerTextoNoVacio(scanner, "Nombre del estudiante:
");
    String fecha = leerTextoNoVacio(scanner, "Fecha (dd/mm/aaaa): ");
    String estado = leerEstado(scanner);

    String registro = nombre + "|" + fecha + "|" + estado;
    guardarRegistro(registro);

    System.out.println("Asistencia registrada correctamente.");
}

private static String leerTextoNoVacio(Scanner scanner, String
mensaje) {
    String texto;
    do {
        System.out.print(mensaje);
        texto = scanner.nextLine().trim();
        if (texto.isEmpty()) {
            System.out.println("El valor no puede estar vacío.
Intente de nuevo.");
        }
    } while (texto.isEmpty());
    return texto;
}

private static String leerEstado(Scanner scanner) {
    String estado;
    do {
        System.out.print("Estado (P = presente, F = falta): ");
        estado = scanner.nextLine().trim().toUpperCase();
        if (!estado.equals("P") && !estado.equals("F")) {
            System.out.println("Estado inválido. Ingrese P o F.");
        }
    }
```



```
    } while (!estado.equals("P") && !estado.equals("F"));
    return estado;
}

private static void guardarRegistro(String registro) {
    File archivo = new File(NOMBRE_ARCHIVO);
    try (BufferedWriter writer = new BufferedWriter(new
FileWriter(archivo, true))) {
        writer.write(registro);
        writer.newLine();
    } catch (IOException e) {
        System.out.println("Error al guardar el registro: " +
e.getMessage());
    }
}

private static void mostrarHistorial() {
    File archivo = new File(NOMBRE_ARCHIVO);
    if (!archivo.exists()) {
        System.out.println("No hay historial de asistencia
registrado.");
        return;
    }

    System.out.println("--- Historial de asistencia ---");
    try (BufferedReader reader = new BufferedReader(new
FileReader(archivo))) {
        String linea;
        int contador = 0;
        while ((linea = reader.readLine()) != null) {
            contador++;
            String[] partes = linea.split("\\|");
            if (partes.length == 3) {
                String nombre = partes[0];
                String fecha = partes[1];
                String estado = partes[2].equals("P") ? "Presente" :
"Falta";
                System.out.printf("%d. %s - %s - %s\n", contador,
nombre, fecha, estado);
            } else {
                System.out.printf("%d. Registro inválido: %s\n",
contador, linea);
            }
        }
        if (contador == 0) {
            System.out.println("No hay registros en el archivo.");
        }
    } catch (IOException e) {
```



```
        System.out.println("Error al leer el historial: " +
e.getMessage());
    }
}

private static void contarFaltas() {
    File archivo = new File(NOMBRE_ARCHIVO);
    if (!archivo.exists()) {
        System.out.println("No hay registros disponibles para contar
faltas.");
        return;
    }

    int faltas = 0;
    try (BufferedReader reader = new BufferedReader(new
FileReader(archivo))) {
        String linea;
        while ((linea = reader.readLine()) != null) {
            String[] partes = linea.split("\\|");
            if (partes.length >= 3 &&
partes[2].equalsIgnoreCase("F")) {
                faltas++;
            }
        }
        System.out.println("Total de faltas registradas: " + faltas);
    } catch (IOException e) {
        System.out.println("Error al contar faltas: " +
e.getMessage());
    }
}
}
```

Código en C++

```
#include <fstream>
#include <iostream>
#include <limits>
#include <sstream>
#include <string>

const std::string NOMBRE_ARCHIVO = "asistencia.txt";

int leerEntero(const std::string &mensaje) {
    int valor;
    while (true) {
        std::cout << mensaje;
        if (std::cin >> valor) {
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
```



```
        return valor;
    }
    std::cout << "Debe ingresar un numero valido." << std::endl;
    std::cin.clear();
    std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
    }
}

std::string leerTextoNoVacio(const std::string &mensaje) {
    std::string texto;
    do {
        std::cout << mensaje;
        std::getline(std::cin, texto);
        if (texto.empty()) {
            std::cout << "El valor no puede estar vacio. Intente de
nuevo." << std::endl;
        }
    } while (texto.empty());
    return texto;
}

std::string leerEstado() {
    std::string estado;
    do {
        std::cout << "Estado (P = presente, F = falta): ";
        std::getline(std::cin, estado);
        for (auto &c : estado) c = toupper(static_cast<unsigned
char>(c));
        if (estado != "P" && estado != "F") {
            std::cout << "Estado invalido. Ingrese P o F." << std::endl;
        }
    } while (estado != "P" && estado != "F");
    return estado;
}

void guardarRegistro(const std::string &registro) {
    std::ofstream archivo(NOMBRE_ARCHIVO, std::ios::app);
    if (!archivo) {
        std::cout << "Error al abrir el archivo para guardar el
registro." << std::endl;
        return;
    }
    archivo << registro << std::endl;
}

void registrarAsistencia() {
    std::string nombre = leerTextoNoVacio("Nombre del estudiante: ");
```



```
std::string fecha = leerTextoNoVacio("Fecha (dd/mm/aaaa): ");
std::string estado = leerEstado();
std::string registro = nombre + "|" + fecha + "|" + estado;
guardarRegistro(registro);
std::cout << "Asistencia registrada correctamente." << std::endl;
}

void mostrarHistorial() {
    std::ifstream archivo(NOMBRE_ARCHIVO);
    if (!archivo) {
        std::cout << "No hay historial de asistencia registrado." <<
std::endl;
        return;
    }

    std::cout << "--- Historial de asistencia ---" << std::endl;
    std::string linea;
    int contador = 0;
    while (std::getline(archivo, linea)) {
        contador++;
        std::stringstream ss(linea);
        std::string nombre, fecha, estado;
        if (std::getline(ss, nombre, '|') && std::getline(ss, fecha, '|')
&& std::getline(ss, estado)) {
            std::string textoEstado = (estado == "P") ? "Presente" :
"Falta";
            std::cout << contador << ". " << nombre << " - " << fecha <<
" - " << textoEstado << std::endl;
        } else {
            std::cout << contador << ". Registro invalido: " << linea <<
std::endl;
        }
    }
    if (contador == 0) {
        std::cout << "No hay registros en el archivo." << std::endl;
    }
}

void contarFaltas() {
    std::ifstream archivo(NOMBRE_ARCHIVO);
    if (!archivo) {
        std::cout << "No hay registros disponibles para contar faltas."
<< std::endl;
        return;
    }

    int faltas = 0;
    std::string linea;
```



```
while (std::getline(archivo, linea)) {
    std::stringstream ss(linea);
    std::string nombre, fecha, estado;
    if (std::getline(ss, nombre, '|') && std::getline(ss, fecha, '|')
    && std::getline(ss, estado)) {
        if (estado == "F") {
            faltas++;
        }
    }
}
std::cout << "Total de faltas registradas: " << faltas << std::endl;
}

void mostrarMenu() {
    std::cout << "=== Control de Asistencia ===" << std::endl;
    std::cout << "1. Registrar asistencia" << std::endl;
    std::cout << "2. Mostrar historial" << std::endl;
    std::cout << "3. Contar faltas" << std::endl;
    std::cout << "4. Salir" << std::endl;
}

int main() {
    int opcion;
    do {
        mostrarMenu();
        opcion = leerEntero("Seleccione una opcion: ");
        switch (opcion) {
            case 1:
                registrarAsistencia();
                break;
            case 2:
                mostrarHistorial();
                break;
            case 3:
                contarFaltas();
                break;
            case 4:
                std::cout << "Saliendo del programa. !Hasta luego!" <<
std::endl;
                break;
            default:
                std::cout << "Opcion invalida. Intente de nuevo." <<
std::endl;
        }
        std::cout << std::endl;
    } while (opcion != 4);
    return 0;
}
```



Casos de pruebas

```
=== Control de Asistencia ===
1. Registrar asistencia
2. Mostrar historial
3. Contar faltas
4. Salir
Seleccione una opción: 1
Nombre del estudiante: Bryan Nuñez
Fecha (dd/mm/aaaa): 22/05/2026
Estado (P = presente, F = falta): P
Asistencia registrada correctamente.

=== Control de Asistencia ===
1. Registrar asistencia
2. Mostrar historial
3. Contar faltas
4. Salir
Seleccione una opción: 2
--- Historial de asistencia ---
1. Bryan Nuñez - 22/05/2026 - Presente

=== Control de Asistencia ===
1. Registrar asistencia
2. Mostrar historial
3. Contar faltas
4. Salir
Seleccione una opción: 3
Total de faltas registradas: 0
```

Ejercicio 7 – Cajero Automático Básico

Objetivo: Simular un cajero automático utilizando archivos.

- Requerimientos:
 - Guardar usuario y saldo.
 - Consultar saldo.
 - Depositar y retirar dinero.
- Aplicar:
 - POO
 - Archivos
 - Métodos

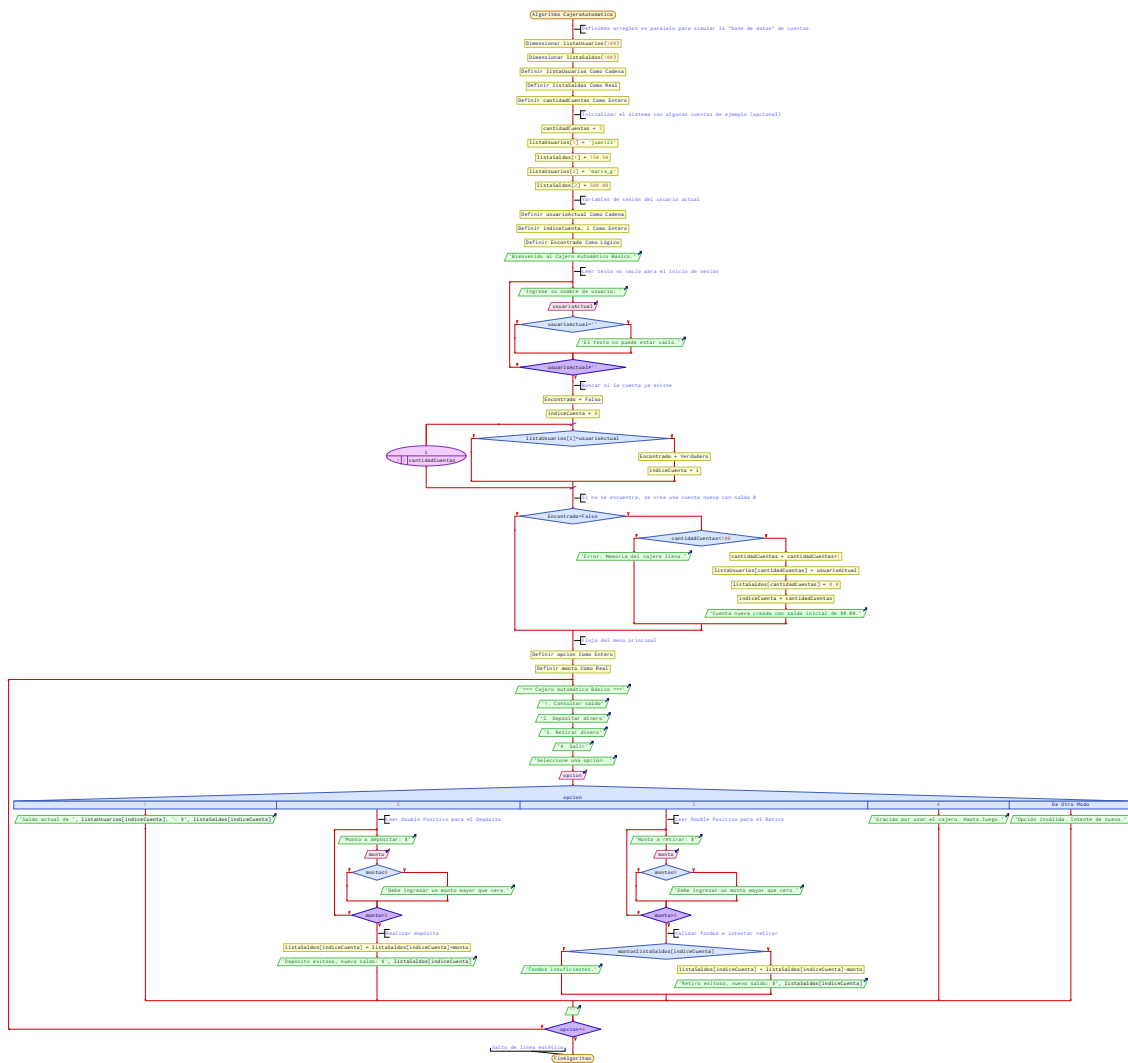
Análisis

Vamos a usar temas aprendidos en clase para poder crear una simulación de un cajero que maneja dinero electrónico y que se guarden los datos de cada usuario registrado.

Diagrama de flujo



UNIVERSIDAD TÉCNICA DE AMBATO
FACULTAD DE INGENIERÍA EN SISTEMAS ELECTRÓNICA E INDUSTRIAL
CARRERA DE Elige un elemento.
CICLO ACADÉMICO: ENERO 2026 – JULIO 2026



Código en Java

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.File;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.util.HashMap;
import java.util.Map;
import java.util.Scanner;

public class Ejercicio7 {

    private static final String ARCHIVO_CUENTAS = "cuentas.txt";

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        Map<String, CuentaUsuario> cuentas = cargarCuentas();
```




```
System.out.println("=== Cajero Automático Básico ===");
String usuario = leerTextoNoVacio(scanner, "Ingrese su nombre de
usuario: ");
CuentaUsuario cuenta = cuentas.getDefault(usuario, new
CuentaUsuario(usuario, 0.0));
cuentas.put(usuario, cuenta);

int opcion;
do {
    mostrarMenu();
    opcion = leerEntero(scanner, "Seleccione una opción: ");
    switch (opcion) {
        case 1:
            System.out.printf("Saldo actual de %s: $%.2f%n",
cuenta.getUsuario(), cuenta.getSaldo());
            break;
        case 2:
            realizarDeposito(scanner, cuenta);
            guardarCuentas(cuentas);
            break;
        case 3:
            realizarRetiro(scanner, cuenta);
            guardarCuentas(cuentas);
            break;
        case 4:
            System.out.println("Gracias por usar el cajero. Hasta
luego.");
            break;
        default:
            System.out.println("Opción inválida. Intente de
nuevo.");
    }
    System.out.println();
} while (opcion != 4);

scanner.close();
}

private static void mostrarMenu() {
    System.out.println("1. Consultar saldo");
    System.out.println("2. Depositar dinero");
    System.out.println("3. Retirar dinero");
    System.out.println("4. Salir");
}

private static int leerEntero(Scanner scanner, String mensaje) {
    while (true) {
        System.out.print(mensaje);
```



```
String linea = scanner.nextLine().trim();
try {
    return Integer.parseInt(linea);
} catch (NumberFormatException e) {
    System.out.println("Debe ingresar un número válido.");
}
}

private static double leerDoublePositivo(Scanner scanner, String
mensaje) {
    while (true) {
        System.out.print(mensaje);
        String linea = scanner.nextLine().trim();
        try {
            double valor = Double.parseDouble(linea);
            if (valor <= 0) {
                System.out.println("Debe ingresar un monto mayor que
cero.");
            } else {
                return valor;
            }
        } catch (NumberFormatException e) {
            System.out.println("Debe ingresar un número válido.");
        }
    }
}

private static String leerTextoNoVacio(Scanner scanner, String
mensaje) {
    while (true) {
        System.out.print(mensaje);
        String texto = scanner.nextLine().trim();
        if (!texto.isEmpty()) {
            return texto;
        }
        System.out.println("El texto no puede estar vacío.");
    }
}

private static void realizarDeposito(Scanner scanner, CuentaUsuario
cuenta) {
    double monto = leerDoublePositivo(scanner, "Monto a depositar:
$");
    cuenta.depositar(monto);
    System.out.printf("Depósito exitoso. Nuevo saldo: $%.2f%n",
cuenta.getSaldo());
}
```



```
private static void realizarRetiro(Scanner scanner, CuentaUsuario
cuenta) {
    double monto = leerDoublePositivo(scanner, "Monto a retirar: $");
    if (cuenta.retirar(monto)) {
        System.out.printf("Retiro exitoso. Nuevo saldo: $%.2f%n",
cuenta.getSaldo());
    } else {
        System.out.println("Fondos insuficientes. Operación
cancelada.");
    }
}

private static Map<String, CuentaUsuario> cargarCuentas() {
    Map<String, CuentaUsuario> cuentas = new HashMap<>();
    File archivo = new File(ARCHIVO_CUENTAS);
    if (!archivo.exists()) {
        return cuentas;
    }

    try (BufferedReader reader = new BufferedReader(new
FileReader(archivo))) {
        String linea;
        while ((linea = reader.readLine()) != null) {
            String[] partes = linea.split("\\|");
            if (partes.length == 2) {
                String usuario = partes[0];
                double saldo = Double.parseDouble(partes[1]);
                cuentas.put(usuario, new CuentaUsuario(usuario,
saldo));
            }
        }
    } catch (IOException | NumberFormatException e) {
        System.out.println("Error al cargar cuentas: " +
e.getMessage());
    }
    return cuentas;
}

private static void guardarCuentas(Map<String, CuentaUsuario>
cuentas) {
    File archivo = new File(ARCHIVO_CUENTAS);
    try (BufferedWriter writer = new BufferedWriter(new
FileWriter(archivo))) {
        for (CuentaUsuario cuenta : cuentas.values()) {
            writer.write(cuenta.toRegistro());
            writer.newLine();
        }
    }
}
```



```
    } catch (IOException e) {  
        System.out.println("Error al guardar cuentas: " +  
e.getMessage());  
    }  
}  
}  
  
class CuentaUsuario {  
  
    private final String usuario;  
    private double saldo;  
  
    public CuentaUsuario(String usuario, double saldo) {  
        this.usuario = usuario;  
        this.saldo = saldo;  
    }  
  
    public String getUsuario() {  
        return usuario;  
    }  
  
    public double getSaldo() {  
        return saldo;  
    }  
  
    public void depositar(double monto) {  
        if (monto > 0) {  
            saldo += monto;  
        }  
    }  
  
    public boolean retirar(double monto) {  
        if (monto > 0 && monto <= saldo) {  
            saldo -= monto;  
            return true;  
        }  
        return false;  
    }  
  
    public String toRegistro() {  
        return usuario + "|" + saldo;  
    }  
}
```

Código en C++

```
#include <fstream>  
#include <iostream>
```



```
#include <sstream>
#include <string>
#include <vector>
#include <limits>

const std::string ARCHIVO_CUENTAS = "cuentas.txt";

class CuentaUsuario {
public:
    CuentaUsuario(const std::string& usuario, double saldo)
        : usuario(usuario), saldo(saldo) {}

    const std::string& getUsuario() const { return usuario; }
    double getSaldo() const { return saldo; }

    void depositar(double monto) {
        if (monto > 0) {
            saldo += monto;
        }
    }

    bool retirar(double monto) {
        if (monto > 0 && monto <= saldo) {
            saldo -= monto;
            return true;
        }
        return false;
    }

    std::string toRegistro() const {
        std::ostringstream out;
        out << usuario << "|" << saldo;
        return out.str();
    }

private:
    std::string usuario;
    double saldo;
};

int leerEntero(const std::string& mensaje) {
    while (true) {
        std::cout << mensaje;
        int valor;
        if (std::cin >> valor) {
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
            return valor;
        }
    }
}
```



```
}
std::cout << "Debe ingresar un número válido." << std::endl;
std::cin.clear();
std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
}
}

double leerDoublePositivo(const std::string& mensaje) {
    while (true) {
        std::cout << mensaje;
        double valor;
        if (std::cin >> valor) {
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
            if (valor > 0) {
                return valor;
            }
            std::cout << "Debe ingresar un monto mayor que cero." <<
std::endl;
        } else {
            std::cout << "Debe ingresar un número válido." << std::endl;
            std::cin.clear();
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
        }
    }
}

std::string leerTextoNoVacio(const std::string& mensaje) {
    while (true) {
        std::cout << mensaje;
        std::string texto;
        std::getline(std::cin, texto);
        if (!texto.empty()) {
            return texto;
        }
        std::cout << "El texto no puede estar vacío." << std::endl;
    }
}

std::vector<CuentaUsuario> cargarCuentas() {
    std::vector<CuentaUsuario> cuentas;
    std::ifstream archivo(ARCHIVO_CUENTAS);
    if (!archivo.is_open()) {
        return cuentas;
    }
}
```



```
std::string linea;
while (std::getline(archivo, linea)) {
    std::istringstream ss(linea);
    std::string usuario;
    std::string saldoStr;
    if (std::getline(ss, usuario, '|') && std::getline(ss, saldoStr))
    {
        try {
            double saldo = std::stod(saldoStr);
            cuentas.emplace_back(usuario, saldo);
        } catch (...) {
            // ignorar línea malformada
        }
    }
}
return cuentas;
}

void guardarCuentas(const std::vector<CuentaUsuario>& cuentas) {
    std::ofstream archivo(ARCHIVO_CUENTAS);
    if (!archivo.is_open()) {
        std::cout << "Error al guardar cuentas." << std::endl;
        return;
    }

    for (const auto& cuenta : cuentas) {
        archivo << cuenta.toRegistro() << '\n';
    }
}

CuentaUsuario* buscarCuenta(std::vector<CuentaUsuario>& cuentas, const
std::string& usuario) {
    for (auto& cuenta : cuentas) {
        if (cuenta.getUsuario() == usuario) {
            return &cuenta;
        }
    }
    return nullptr;
}

void mostrarMenu() {
    std::cout << "=== Cajero Automático Básico ===" << std::endl;
    std::cout << "1. Consultar saldo" << std::endl;
    std::cout << "2. Depositar dinero" << std::endl;
    std::cout << "3. Retirar dinero" << std::endl;
    std::cout << "4. Salir" << std::endl;
}
```



```
int main() {
    std::vector<CuentaUsuario> cuentas = cargarCuentas();
    std::cout << "Bienvenido al Cajero Automático Básico." << std::endl;
    std::string usuario = leerTextoNoVacio("Ingrese su nombre de usuario:
");

    CuentaUsuario* cuenta = buscarCuenta(cuentas, usuario);
    if (cuenta == nullptr) {
        cuentas.emplace_back(usuario, 0.0);
        cuenta = &cuentas.back();
        std::cout << "Cuenta nueva creada con saldo inicial de $0.00." <<
std::endl;
    }

    int opcion;
    do {
        mostrarMenu();
        opcion = leerEntero("Seleccione una opción: ");
        switch (opcion) {
            case 1:
                std::cout << "Saldo actual de " << cuenta->getUsuario()
<< ": $" << cuenta->getSaldo() << std::endl;
                break;
            case 2: {
                double monto = leerDoublePositivo("Monto a depositar:
$");

                cuenta->depositar(monto);
                guardarCuentas(cuentas);
                std::cout << "Depósito exitoso, nuevo saldo: $" <<
cuenta->getSaldo() << std::endl;
                break;
            }
            case 3: {
                double monto = leerDoublePositivo("Monto a retirar: $");
                if (cuenta->retirar(monto)) {
                    guardarCuentas(cuentas);
                    std::cout << "Retiro exitoso, nuevo saldo: $" <<
cuenta->getSaldo() << std::endl;
                } else {
                    std::cout << "Fondos insuficientes." << std::endl;
                }
                break;
            }
            case 4:
                std::cout << "Gracias por usar el cajero. Hasta luego."
<< std::endl;
                break;
            default:
```




```
        std::cout << "Opción inválida. Intente de nuevo." <<
std::endl;
    }
    std::cout << std::endl;
} while (opcion != 4);

return 0;
}
```

Casos de pruebas

```
=== Cajero Automatico Básico ===
Ingrese su nombre de usuario: Kevin
1. Consultar saldo
2. Depositar dinero
4. Salir
Seleccione una opción: 1
Saldo actual de Kevin: $80.00

1. Consultar saldo
3. Retirar dinero
4. Salir
Seleccione una opción: 4
Gracias por usar el cajero. Hasta luego.
```

Ejercicio 8 - Biblioteca Virtual

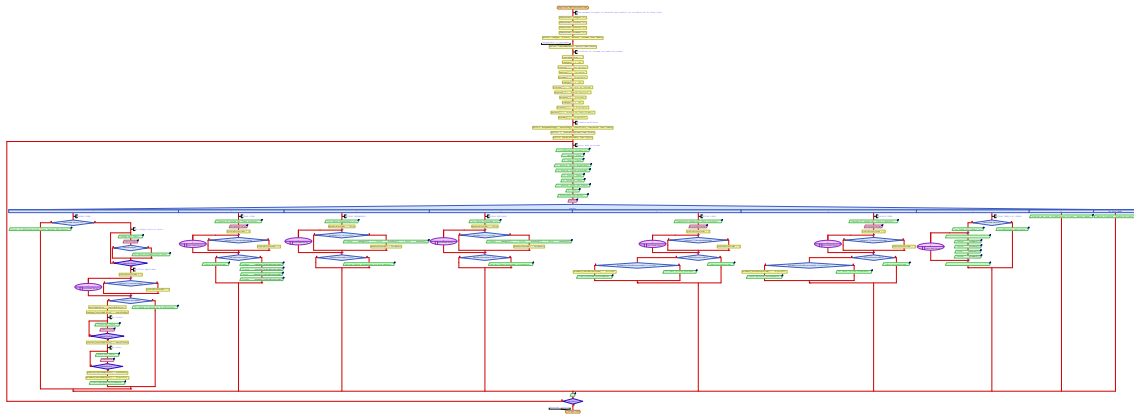
Objetivo: Registrar libros en un archivo.

- Requerimientos:
 - Guardar código, título, autor y estado.
 - Buscar libros.
 - Mostrar disponibles y prestados.
- Aplicar:
 - POO
 - Archivos
 - Métodos

Análisis

Vamos a crear un programa similar al anterior ejercicio, pero enfocado a un ambiente virtual donde guardan las entradas y salidas de libros.

Diagrama de flujo



Código en Java

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.File;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Ejercicio8 {

    private static final String ARCHIVO_LIBROS = "biblioteca.txt";
    private static List<Libro> libros = new ArrayList<>();

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        cargarLibros();

        int opcion;
        do {
            mostrarMenu();
            opcion = leerEntero(scanner, "Seleccione una opción: ");
            switch (opcion) {
                case 1:
                    agregarLibro(scanner);
                    break;
                case 2:
                    buscarLibro(scanner);
                    break;
                case 3:
                    mostrarDisponibles();
                    break;
                case 4:
                    mostrarPrestados();
                    break;
            }
        } while (opcion != 0);
    }
}
```



```
        break;
    case 5:
        prestarLibro(scanner);
        break;
    case 6:
        devolverLibro(scanner);
        break;
    case 7:
        mostrarTodos();
        break;
    case 8:
        System.out.println("Gracias por usar la Biblioteca
Virtual. ¡Hasta luego!");
        break;
    default:
        System.out.println("Opción inválida. Intente de
nuevo.");
    }
    System.out.println();
} while (opcion != 8);

guardarLibros();
scanner.close();
}

private static void mostrarMenu() {
    System.out.println("=== Biblioteca Virtual ===");
    System.out.println("1. Agregar libro");
    System.out.println("2. Buscar libro");
    System.out.println("3. Mostrar libros disponibles");
    System.out.println("4. Mostrar libros prestados");
    System.out.println("5. Prestar libro");
    System.out.println("6. Devolver libro");
    System.out.println("7. Mostrar todos los libros");
    System.out.println("8. Salir");
}

private static int leerEntero(Scanner scanner, String mensaje) {
    while (true) {
        System.out.print(mensaje);
        String linea = scanner.nextLine().trim();
        try {
            return Integer.parseInt(linea);
        } catch (NumberFormatException e) {
            System.out.println("Debe ingresar un número válido.");
        }
    }
}
```



```
private static String leerTextoNoVacio(Scanner scanner, String
mensaje) {
    while (true) {
        System.out.print(mensaje);
        String texto = scanner.nextLine().trim();
        if (!texto.isEmpty()) {
            return texto;
        }
        System.out.println("El texto no puede estar vacío.");
    }
}

private static void agregarLibro(Scanner scanner) {
    String codigo = leerTextoNoVacio(scanner, "Código del libro: ");
    if (buscarPorCodigo(codigo) != null) {
        System.out.println("El código ya existe en la biblioteca.");
        return;
    }
    String titulo = leerTextoNoVacio(scanner, "Título del libro: ");
    String autor = leerTextoNoVacio(scanner, "Autor del libro: ");
    Libro libro = new Libro(codigo, titulo, autor, "disponible");
    libros.add(libro);
    guardarLibros();
    System.out.println("Libro agregado exitosamente.");
}

private static void buscarLibro(Scanner scanner) {
    String codigo = leerTextoNoVacio(scanner, "Ingrese el código del
libro a buscar: ");
    Libro libro = buscarPorCodigo(codigo);
    if (libro != null) {
        System.out.println(libro);
    } else {
        System.out.println("Libro no encontrado.");
    }
}

private static Libro buscarPorCodigo(String codigo) {
    for (Libro libro : libros) {
        if (libro.getCodigo().equalsIgnoreCase(codigo)) {
            return libro;
        }
    }
    return null;
}

private static void mostrarDisponibles() {
```



```
List<Libro> disponibles = new ArrayList<>();
for (Libro libro : libros) {
    if (libro.isDisponible()) {
        disponibles.add(libro);
    }
}

if (disponibles.isEmpty()) {
    System.out.println("No hay libros disponibles en este
momento.");
} else {
    System.out.println("=== Libros Disponibles ===");
    for (int i = 0; i < disponibles.size(); i++) {
        System.out.printf("%d. %s\n", i + 1, disponibles.get(i));
    }
}

private static void mostrarPrestados() {
    List<Libro> prestados = new ArrayList<>();
    for (Libro libro : libros) {
        if (!libro.isDisponible()) {
            prestados.add(libro);
        }
    }

    if (prestados.isEmpty()) {
        System.out.println("No hay libros prestados actualmente.");
    } else {
        System.out.println("=== Libros Prestados ===");
        for (int i = 0; i < prestados.size(); i++) {
            System.out.printf("%d. %s\n", i + 1, prestados.get(i));
        }
    }
}

private static void mostrarTodos() {
    if (libros.isEmpty()) {
        System.out.println("La biblioteca está vacía.");
    } else {
        System.out.println("=== Todos los Libros ===");
        for (int i = 0; i < libros.size(); i++) {
            System.out.printf("%d. %s\n", i + 1, libros.get(i));
        }
    }
}

private static void prestarLibro(Scanner scanner) {
```



```
String codigo = leerTextoNoVacio(scanner, "Ingrese el código del
libro a prestar: ");
Libro libro = buscarPorCodigo(codigo);
if (libro == null) {
    System.out.println("Libro no encontrado.");
} else if (!libro.isDisponible()) {
    System.out.println("El libro ya está prestado.");
} else {
    libro.prestar();
    guardarLibros();
    System.out.println("Libro prestado exitosamente.");
}
}

private static void devolverLibro(Scanner scanner) {
    String codigo = leerTextoNoVacio(scanner, "Ingrese el código del
libro a devolver: ");
    Libro libro = buscarPorCodigo(codigo);
    if (libro == null) {
        System.out.println("Libro no encontrado.");
    } else if (libro.isDisponible()) {
        System.out.println("El libro ya está disponible.");
    } else {
        libro.devolver();
        guardarLibros();
        System.out.println("Libro devuelto exitosamente.");
    }
}

private static void cargarLibros() {
    File archivo = new File(ARCHIVO_LIBROS);
    if (!archivo.exists()) {
        return;
    }

    try (BufferedReader reader = new BufferedReader(new
FileReader(archivo))) {
        String linea;
        while ((linea = reader.readLine()) != null) {
            String[] partes = linea.split("\\|");
            if (partes.length == 4) {
                String codigo = partes[0];
                String titulo = partes[1];
                String autor = partes[2];
                String estado = partes[3];
                libros.add(new Libro(codigo, titulo, autor, estado));
            }
        }
    }
}
```



```
        } catch (IOException e) {
            System.out.println("Error al cargar libros: " +
e.getMessage());
        }
    }

    private static void guardarLibros() {
        try (BufferedWriter writer = new BufferedWriter(new
FileWriter(ARCHIVO_LIBROS))) {
            for (Libro libro : libros) {
                writer.write(libro.toRegistro());
                writer.newLine();
            }
        } catch (IOException e) {
            System.out.println("Error al guardar libros: " +
e.getMessage());
        }
    }
}

class Libro {

    private final String codigo;
    private final String titulo;
    private final String autor;
    private String estado;

    public Libro(String codigo, String titulo, String autor, String
estado) {
        this.codigo = codigo;
        this.titulo = titulo;
        this.autor = autor;
        this.estado = estado;
    }

    public String getCodigo() {
        return codigo;
    }

    public boolean isDisponible() {
        return estado.equalsIgnoreCase("disponible");
    }

    public void prestar() {
        estado = "prestado";
    }

    public void devolver() {
```



```
        estado = "disponible";
    }

    public String toRegistro() {
        return codigo + "|" + titulo + "|" + autor + "|" + estado;
    }

    @Override
    public String toString() {
        return String.format("[%s] %s - %s (%s)", codigo, titulo, autor,
estado);
    }
}
```

Código en C++

```
#include <fstream>
#include <iostream>
#include <sstream>
#include <string>
#include <vector>
#include <limits>

const std::string ARCHIVO_LIBROS = "biblioteca.txt";

class Libro {
public:
    Libro(const std::string &codigo, const std::string &titulo, const
std::string &autor, const std::string &estado)
        : codigo(codigo), titulo(titulo), autor(autor), estado(estado) {}

    const std::string &getCodigo() const { return codigo; }
    const std::string &getTitulo() const { return titulo; }
    const std::string &getAutor() const { return autor; }
    const std::string &getEstado() const { return estado; }

    bool estaDisponible() const {
        return estado == "disponible";
    }

    void prestar() {
        estado = "prestado";
    }

    void devolver() {
        estado = "disponible";
    }
}
```




```
std::string toRegistro() const {
    std::ostringstream out;
    out << codigo << "|" << titulo << "|" << autor << "|" << estado;
    return out.str();
}

std::string toString() const {
    std::ostringstream out;
    out << "Codigo: " << codigo << "\n"
        << "Titulo: " << titulo << "\n"
        << "Autor: " << autor << "\n"
        << "Estado: " << estado;
    return out.str();
}

private:
    std::string codigo;
    std::string titulo;
    std::string autor;
    std::string estado;
};

int leerEntero(const std::string &mensaje) {
    while (true) {
        std::cout << mensaje;
        int valor;
        if (std::cin >> valor) {
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
            return valor;
        }
        std::cout << "Debe ingresar un número válido." << std::endl;
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
    }
}

std::string leerTextoNoVacio(const std::string &mensaje) {
    while (true) {
        std::cout << mensaje;
        std::string texto;
        std::getline(std::cin, texto);
        if (!texto.empty()) {
            return texto;
        }
        std::cout << "El texto no puede estar vacío." << std::endl;
    }
}
```



```
}  
}  
  
std::vector<Libro> cargarLibros() {  
    std::vector<Libro> libros;  
    std::ifstream archivo(ARCHIVO_LIBROS);  
    if (!archivo.is_open()) {  
        return libros;  
    }  
  
    std::string linea;  
    while (std::getline(archivo, linea)) {  
        std::istringstream ss(linea);  
        std::string codigo, titulo, autor, estado;  
        if (std::getline(ss, codigo, '|') && std::getline(ss, titulo,  
'|') && std::getline(ss, autor, '|') && std::getline(ss, estado)) {  
            libros.emplace_back(codigo, titulo, autor, estado);  
        }  
    }  
    return libros;  
}  
  
void guardarLibros(const std::vector<Libro> &libros) {  
    std::ofstream archivo(ARCHIVO_LIBROS);  
    if (!archivo.is_open()) {  
        std::cout << "Error al guardar libros." << std::endl;  
        return;  
    }  
    for (const auto &libro : libros) {  
        archivo << libro.toRegistro() << "\n";  
    }  
}  
  
Libro *buscarLibro(std::vector<Libro> &libros, const std::string &codigo)  
{  
    for (auto &libro : libros) {  
        if (libro.getCodigo() == codigo) {  
            return &libro;  
        }  
    }  
    return nullptr;  
}  
  
void mostrarMenu() {  
    std::cout << "=== Biblioteca Virtual ===" << std::endl;  
    std::cout << "1. Agregar libro" << std::endl;  
    std::cout << "2. Buscar libro" << std::endl;  
    std::cout << "3. Mostrar libros disponibles" << std::endl;
```



```
std::cout << "4. Mostrar libros prestados" << std::endl;
std::cout << "5. Prestar libro" << std::endl;
std::cout << "6. Devolver libro" << std::endl;
std::cout << "7. Mostrar todos los libros" << std::endl;
std::cout << "8. Salir" << std::endl;
}

void agregarLibro(std::vector<Libro> &libros) {
    std::string codigo = leerTextoNoVacio("Codigo del libro: ");
    if (buscarLibro(libros, codigo) != nullptr) {
        std::cout << "El codigo ya existe en la biblioteca." <<
std::endl;
        return;
    }
    std::string titulo = leerTextoNoVacio("Titulo del libro: ");
    std::string autor = leerTextoNoVacio("Autor del libro: ");
    libros.emplace_back(codigo, titulo, autor, "disponible");
    guardarLibros(libros);
    std::cout << "Libro agregado exitosamente." << std::endl;
}

void buscarLibro(const std::vector<Libro> &libros) {
    std::string codigo = leerTextoNoVacio("Ingrese el codigo del libro a
buscar: ");
    for (const auto &libro : libros) {
        if (libro.getCodigo() == codigo) {
            std::cout << libro.toString() << std::endl;
            return;
        }
    }
    std::cout << "Libro no encontrado." << std::endl;
}

void mostrarDisponibles(const std::vector<Libro> &libros) {
    bool encontrado = false;
    std::cout << "=== Libros Disponibles ===" << std::endl;
    for (const auto &libro : libros) {
        if (libro.estaDisponible()) {
            std::cout << libro.toString() << std::endl << std::endl;
            encontrado = true;
        }
    }
    if (!encontrado) {
        std::cout << "No hay libros disponibles en este momento." <<
std::endl;
    }
}
```



```
void mostrarPrestados(const std::vector<Libro> &libros) {
    bool encontrado = false;
    std::cout << "=== Libros Prestados ===" << std::endl;
    for (const auto &libro : libros) {
        if (!libro.estaDisponible()) {
            std::cout << libro.toString() << std::endl << std::endl;
            encontrado = true;
        }
    }
    if (!encontrado) {
        std::cout << "No hay libros prestados actualmente." << std::endl;
    }
}

void prestarLibro(std::vector<Libro> &libros) {
    std::string codigo = leerTextoNoVacio("Ingrese el código del libro a
prestar: ");
    Libro *libro = buscarLibro(libros, codigo);
    if (libro == nullptr) {
        std::cout << "Libro no encontrado." << std::endl;
    } else if (!libro->estaDisponible()) {
        std::cout << "El libro ya esta prestado." << std::endl;
    } else {
        libro->prestar();
        guardarLibros(libros);
        std::cout << "Libro prestado exitosamente." << std::endl;
    }
}

void devolverLibro(std::vector<Libro> &libros) {
    std::string codigo = leerTextoNoVacio("Ingrese el código del libro a
devolver: ");
    Libro *libro = buscarLibro(libros, codigo);
    if (libro == nullptr) {
        std::cout << "Libro no encontrado." << std::endl;
    } else if (libro->estaDisponible()) {
        std::cout << "El libro ya está disponible." << std::endl;
    } else {
        libro->devolver();
        guardarLibros(libros);
        std::cout << "Libro devuelto exitosamente." << std::endl;
    }
}

void mostrarTodos(const std::vector<Libro> &libros) {
    if (libros.empty()) {
        std::cout << "La biblioteca esta vacia." << std::endl;
        return;
    }
}
```



```
}  
std::cout << "=== Todos los Libros ===" << std::endl;  
for (const auto &libro : libros) {  
    std::cout << libro.toString() << std::endl << std::endl;  
}  
}  
  
int main() {  
    std::vector<Libro> libros = cargarLibros();  
    int opcion;  
    do {  
        mostrarMenu();  
        opcion = leerEntero("Seleccione una opcion: ");  
        switch (opcion) {  
            case 1:  
                agregarLibro(libros);  
                break;  
            case 2:  
                buscarLibro(libros);  
                break;  
            case 3:  
                mostrarDisponibles(libros);  
                break;  
            case 4:  
                mostrarPrestados(libros);  
                break;  
            case 5:  
                prestarLibro(libros);  
                break;  
            case 6:  
                devolverLibro(libros);  
                break;  
            case 7:  
                mostrarTodos(libros);  
                break;  
            case 8:  
                std::cout << "Gracias por usar la Biblioteca Virtual.  
❖Hasta luego!" << std::endl;  
                break;  
            default:  
                std::cout << "Opcion invalida. Intente de nuevo." <<  
std::endl;  
                }  
                std::cout << std::endl;  
            } while (opcion != 8);  
            return 0;  
        }
```



Casos de pruebas

```
=== Biblioteca Virtual ===
1. Agregar libro
2. Buscar libro
3. Mostrar libros disponibles
4. Mostrar libros prestados
5. Prestar libro
6. Devolver libro
7. Mostrar todos los libros
8. Salir
Seleccione una opción: 3
No hay libros disponibles en este momento.

=== Biblioteca Virtual ===
1. Agregar libro
2. Buscar libro
3. Mostrar libros disponibles
4. Mostrar libros prestados
5. Prestar libro
6. Devolver libro
7. Mostrar todos los libros
8. Salir
Seleccione una opción: 6
Ingrese el código del libro a devolver: 1876
Libro devuelto exitosamente.
```

Ejercicio 9 – Sistema de Ventas

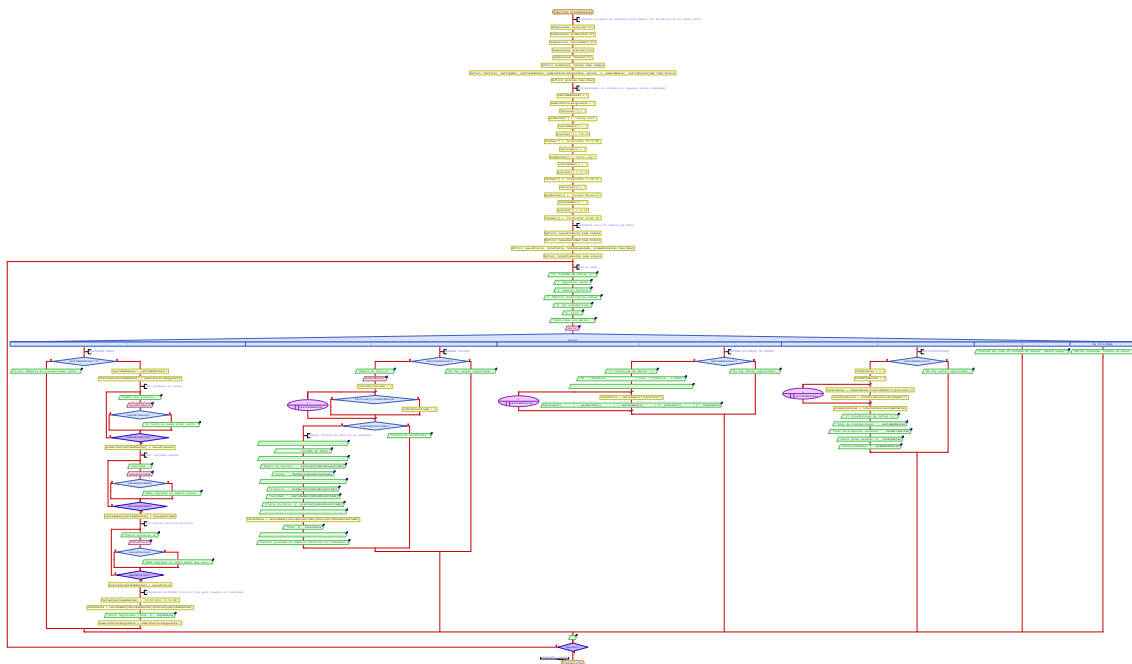
Objetivo: Crear un sistema básico de ventas.

- Requerimientos:
 - Registrar producto, cantidad, precio y total.
 - Generar factura.
 - Mostrar historial de ventas.
- Aplicar:
 - POO
 - Métodos
 - Archivos

Análisis

Vamos a generar un programa basándonos en programas anteriores que ya se realizó para tomar sus bases y crear ahí un sistema de control de ventas.

Diagrama de flujo



Código en Java

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.File;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Ejercicio9 {

    private static final String ARCHIVO_VENTAS = "ventas.txt";
    private static final String ARCHIVO_FACTURAS = "facturas.txt";
    private static List<Venta> ventas = new ArrayList<>();
    private static int numeroFactura = 1;

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        cargarVentas();

        int opcion;
        do {
            mostrarMenu();
            opcion = leerEntero(scanner, "Seleccione una opción: ");
            switch (opcion) {
```



```
        case 1:
            registrarVenta(scanner);
            break;
        case 2:
            generarFactura(scanner);
            break;
        case 3:
            mostrarHistorialVentas();
            break;
        case 4:
            mostrarEstadisticas();
            break;
        case 5:
            System.out.println("Gracias por usar el Sistema de
Ventas. ¡Hasta luego!");
            break;
        default:
            System.out.println("Opción inválida. Intente de
nuevo.");
    }
    System.out.println();
} while (opcion != 5);

guardarVentas();
scanner.close();
}

private static void mostrarMenu() {
    System.out.println("=== Sistema de Ventas ===");
    System.out.println("1. Registrar venta");
    System.out.println("2. Generar factura");
    System.out.println("3. Mostrar historial de ventas");
    System.out.println("4. Ver estadísticas");
    System.out.println("5. Salir");
}

private static int leerEntero(Scanner scanner, String mensaje) {
    while (true) {
        System.out.print(mensaje);
        String linea = scanner.nextLine().trim();
        try {
            return Integer.parseInt(linea);
        } catch (NumberFormatException e) {
            System.out.println("Debe ingresar un número válido.");
        }
    }
}
```




```
private static double leerDoublePositivo(Scanner scanner, String
mensaje) {
    while (true) {
        System.out.print(mensaje);
        String linea = scanner.nextLine().trim();
        try {
            double valor = Double.parseDouble(linea);
            if (valor <= 0) {
                System.out.println("Debe ingresar un valor mayor que
cero.");
            } else {
                return valor;
            }
        } catch (NumberFormatException e) {
            System.out.println("Debe ingresar un número válido.");
        }
    }
}

private static String leerTextoNoVacio(Scanner scanner, String
mensaje) {
    while (true) {
        System.out.print(mensaje);
        String texto = scanner.nextLine().trim();
        if (!texto.isEmpty()) {
            return texto;
        }
        System.out.println("El texto no puede estar vacío.");
    }
}

private static void registrarVenta(Scanner scanner) {
    String producto = leerTextoNoVacio(scanner, "Nombre del producto:
");
    int cantidad = leerEntero(scanner, "Cantidad: ");
    double precio = leerDoublePositivo(scanner, "Precio unitario:
$");

    Venta venta = new Venta(numeroFactura, producto, cantidad,
precio);
    ventas.add(venta);
    numeroFactura++;

    guardarVentas();
    System.out.printf("Venta registrada. Total: $%.2f%n",
venta.calcularTotal());
}
```



```
private static void generarFactura(Scanner scanner) {
    if (ventas.isEmpty()) {
        System.out.println("No hay ventas registradas.");
        return;
    }

    System.out.println("Seleccione el número de factura a generar:");
    int numero = leerEntero(scanner, "Número de factura: ");

    Venta venta = buscarVentaPorNumero(numero);
    if (venta != null) {
        String factura = venta.generarFactura();
        guardarFactura(factura);
        System.out.println(factura);
        System.out.println("Factura guardada en " +
ARCHIVO_FACTURAS);
    } else {
        System.out.println("Factura no encontrada.");
    }
}

private static Venta buscarVentaPorNumero(int numero) {
    for (Venta venta : ventas) {
        if (venta.getNumeroFactura() == numero) {
            return venta;
        }
    }
    return null;
}

private static void mostrarHistorialVentas() {
    if (ventas.isEmpty()) {
        System.out.println("No hay ventas registradas.");
        return;
    }

    System.out.println("=== Historial de Ventas ===");
    System.out.printf("%-5s %-25s %-10s %-12s %-12s%n", "ID",
"Producto", "Cantidad", "Precio Unit.", "Total");
    System.out.println("-".repeat(70));

    for (Venta venta : ventas) {
        System.out.printf("%-5d %-25s %-10d $%-11.2f $%-11.2f%n",
            venta.getNumeroFactura(),
            truncar(venta.getProducto(), 23),
            venta.getCantidad(),
            venta.getPrecio(),
            venta.calcularTotal());
    }
}
```



```
    }  
}  
  
private static void mostrarEstadisticas() {  
    if (ventas.isEmpty()) {  
        System.out.println("No hay ventas registradas.");  
        return;  
    }  
  
    double totalVentas = 0;  
    int totalProductos = 0;  
  
    for (Venta venta : ventas) {  
        totalVentas += venta.calcularTotal();  
        totalProductos += venta.getCantidad();  
    }  
  
    double promedio = totalVentas / ventas.size();  
  
    System.out.println("=== Estadísticas de Ventas ===");  
    System.out.printf("Total de transacciones: %d%n", ventas.size());  
    System.out.printf("Total de productos vendidos: %d%n",  
totalProductos);  
    System.out.printf("Monto total vendido: $%.2f%n", totalVentas);  
    System.out.printf("Venta promedio: $%.2f%n", promedio);  
}  
  
private static String truncar(String texto, int longitud) {  
    if (texto.length() <= longitud) {  
        return texto;  
    }  
    return texto.substring(0, longitud - 2) + "..";  
}  
  
private static void guardarFactura(String factura) {  
    try (BufferedWriter writer = new BufferedWriter(new  
FileWriter(ARCHIVO_FACTURAS, true))) {  
        writer.write(factura);  
        writer.write("\n" + "=".repeat(50) + "\n");  
    } catch (IOException e) {  
        System.out.println("Error al guardar factura: " +  
e.getMessage());  
    }  
}  
  
private static void cargarVentas() {  
    File archivo = new File(ARCHIVO_VENTAS);  
    if (!archivo.exists()) {
```



```
        return;
    }

    try (BufferedReader reader = new BufferedReader(new
FileReader(archivo))) {
        String linea;
        while ((linea = reader.readLine()) != null) {
            String[] partes = linea.split("\\|");
            if (partes.length == 5) {
                int numero = Integer.parseInt(partes[0]);
                String producto = partes[1];
                int cantidad = Integer.parseInt(partes[2]);
                double precio = Double.parseDouble(partes[3]);
                String fecha = partes[4];

                Venta venta = new Venta(numero, producto, cantidad,
precio);

                venta.setFecha(fecha);
                ventas.add(venta);

                if (numero >= numeroFactura) {
                    numeroFactura = numero + 1;
                }
            }
        }
    } catch (IOException | NumberFormatException e) {
        System.out.println("Error al cargar ventas: " +
e.getMessage());
    }
}

private static void guardarVentas() {
    try (BufferedWriter writer = new BufferedWriter(new
FileWriter(ARCHIVO_VENTAS))) {
        for (Venta venta : ventas) {
            writer.write(venta.toRegistro());
            writer.newLine();
        }
    } catch (IOException e) {
        System.out.println("Error al guardar ventas: " +
e.getMessage());
    }
}

class Venta {

    private final int numeroFactura;
```



```
private final String producto;
private final int cantidad;
private final double precio;
private String fecha;

public Venta(int numeroFactura, String producto, int cantidad, double
precio) {
    this.numeroFactura = numeroFactura;
    this.producto = producto;
    this.cantidad = cantidad;
    this.precio = precio;
    this.fecha = obtenerFechaActual();
}

public int getNumeroFactura() {
    return numeroFactura;
}

public String getProducto() {
    return producto;
}

public int getCantidad() {
    return cantidad;
}

public double getPrecio() {
    return precio;
}

public double calcularTotal() {
    return cantidad * precio;
}

public void setFecha(String fecha) {
    this.fecha = fecha;
}

private String obtenerFechaActual() {
    DateTimeFormatter formatter =
DateTimeFormatter.ofPattern("dd/MM/yyyy HH:mm:ss");
    return LocalDateTime.now().format(formatter);
}

public String generarFactura() {
    StringBuilder factura = new StringBuilder();
    factura.append("┌┐
└┘\n");
```



```
factura.append("|| FACTURA DE VENTA\n");
factura.append("||\n");
factura.append(String.format("Número de Factura: %d\n",
numeroFactura));
factura.append(String.format("Fecha: %s\n", fecha));
factura.append("-----\n");
factura.append(String.format("Producto: %s\n", producto));
factura.append(String.format("Cantidad: %d\n", cantidad));
factura.append(String.format("Precio Unitario: $%.2f\n",
precio));
factura.append("-----\n");
factura.append(String.format("TOTAL: $%.2f\n", calcularTotal()));
factura.append("=====\n");
return factura.toString();
}

public String toRegistro() {
    return numeroFactura + "|" + producto + "|" + cantidad + "|" +
precio + "|" + fecha;
}
}
```

Código en C++

```
#include <fstream>
#include <iostream>
#include <sstream>
#include <string>
#include <vector>
#include <limits>
#include <ctime>

const std::string ARCHIVO_VENTAS = "ventas.txt";
const std::string ARCHIVO_FACTURAS = "facturas.txt";

class Venta {
public:
    Venta(int numeroFactura, const std::string &producto, int cantidad,
double precio)
        : numeroFactura(numeroFactura), producto(producto),
cantidad(cantidad), precio(precio) {
        fecha = obtenerFechaActual();
    }
}
```



```
int getNumeroFactura() const { return numeroFactura; }
const std::string &getProducto() const { return producto; }
int getCantidad() const { return cantidad; }
double getPrecio() const { return precio; }
const std::string &getFecha() const { return fecha; }

void setFecha(const std::string &fechaValue) {
    fecha = fechaValue;
}

double calcularTotal() const {
    return cantidad * precio;
}

std::string generarFactura() const {
    std::ostringstream out;
    out << "+-----+\n";
    out << "❖          FACTURA DE VENTA\n";
    out << "+-----+\n";
    out << "Número de Factura: " << numeroFactura << "\n";
    out << "Fecha: " << fecha << "\n";
    out << "-----\n";
    out << "Producto: " << producto << "\n";
    out << "Cantidad: " << cantidad << "\n";
    out << "Precio Unitario: $" << precio << "\n";
    out << "-----\n";
    out << "TOTAL: $" << calcularTotal() << "\n";
    out << "-----\n";
    return out.str();
}

std::string toRegistro() const {
    std::ostringstream out;
    out << numeroFactura << "|" << producto << "|" << cantidad << "|"
<< precio << "|" << fecha;
    return out.str();
}

private:
    int numeroFactura;
    std::string producto;
    int cantidad;
    double precio;
    std::string fecha;

    std::string obtenerFechaActual() const {
        std::time_t now = std::time(nullptr);
        char buffer[32];
```



```
std::tm localTime;
#ifdef _WIN32 || defined(_WIN64)
    localtime_s(&localTime, &now);
#else
    localtime_r(&now, &localTime);
#endif
std::strftime(buffer, sizeof(buffer), "%d/%m/%Y %H:%M:%S",
&localTime);
return std::string(buffer);
}
};

int leerEntero(const std::string &mensaje) {
    while (true) {
        std::cout << mensaje;
        int valor;
        if (std::cin >> valor) {
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
            return valor;
        }
        std::cout << "Debe ingresar un número válido." << std::endl;
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
    }
}

double leerDoublePositivo(const std::string &mensaje) {
    while (true) {
        std::cout << mensaje;
        double valor;
        if (std::cin >> valor) {
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
            if (valor > 0) {
                return valor;
            }
            std::cout << "Debe ingresar un valor mayor que cero." <<
std::endl;
        } else {
            std::cout << "Debe ingresar un número válido." << std::endl;
            std::cin.clear();
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
        }
    }
}
```




```
std::string leerTextoNoVacio(const std::string &mensaje) {
    while (true) {
        std::cout << mensaje;
        std::string texto;
        std::getline(std::cin, texto);
        if (!texto.empty()) {
            return texto;
        }
        std::cout << "El texto no puede estar vacío." << std::endl;
    }
}

void guardarFactura(const std::string &factura) {
    std::ofstream archivo(ARCHIVO_FACTURAS, std::ios::app);
    if (!archivo) {
        std::cout << "Error al guardar factura." << std::endl;
        return;
    }
    archivo << factura << "\n" << std::string(50, '-') << "\n";
}

std::vector<Venta> cargarVentas(int &numeroFactura) {
    std::vector<Venta> ventas;
    std::ifstream archivo(ARCHIVO_VENTAS);
    if (!archivo) {
        return ventas;
    }

    std::string linea;
    while (std::getline(archivo, linea)) {
        std::istringstream ss(linea);
        std::string numeroStr, producto, cantidadStr, precioStr, fecha;

        if (std::getline(ss, numeroStr, '|') && std::getline(ss,
producto, '|') &&
            std::getline(ss, cantidadStr, '|') && std::getline(ss,
precioStr, '|') &&
            std::getline(ss, fecha)) {

            try {
                int numero = std::stoi(numeroStr);
                int cantidad = std::stoi(cantidadStr);
                double precio = std::stod(precioStr);
                Venta venta(numero, producto, cantidad, precio);
                venta.setFecha(fecha);
                ventas.push_back(venta);
                if (numero >= numeroFactura) {
```



```
        numeroFactura = numero + 1;
    }
} catch (...) {
    // ignorar línea malformada
}
}

return ventas;
}

void guardarVentas(const std::vector<Venta> &ventas) {
    std::ofstream archivo(ARCHIVO_VENTAS);
    if (!archivo) {
        std::cout << "Error al guardar ventas." << std::endl;
        return;
    }
    for (const auto &venta : ventas) {
        archivo << venta.toRegistro() << "\n";
    }
}

Venta *buscarVentaPorNumero(std::vector<Venta> &ventas, int numero) {
    for (auto &venta : ventas) {
        if (venta.getNumeroFactura() == numero) {
            return &venta;
        }
    }
    return nullptr;
}

std::string truncar(const std::string &texto, size_t longitud) {
    if (texto.size() <= longitud) {
        return texto;
    }
    return texto.substr(0, longitud - 2) + "..";
}

void mostrarMenu() {
    std::cout << "=== Sistema de Ventas ===" << std::endl;
    std::cout << "1. Registrar venta" << std::endl;
    std::cout << "2. Generar factura" << std::endl;
    std::cout << "3. Mostrar historial de ventas" << std::endl;
    std::cout << "4. Ver estadísticas" << std::endl;
    std::cout << "5. Salir" << std::endl;
}

void mostrarHistorialVentas(const std::vector<Venta> &ventas) {
```



```
if (ventas.empty()) {
    std::cout << "No hay ventas registradas." << std::endl;
    return;
}

std::cout << "=== Historial de Ventas ===" << std::endl;
std::cout << std::left << std::setw(5) << "ID" << std::setw(25) <<
"Producto"
        << std::setw(10) << "Cantidad" << std::setw(14) << "Precio
Unit." << std::setw(14)
        << "Total" << std::endl;
std::cout << std::string(70, '-') << std::endl;

for (const auto &venta : ventas) {
    std::cout << std::left << std::setw(5) <<
venta.getNumeroFactura()
        << std::setw(25) << truncar(venta.getProducto(), 23)
        << std::setw(10) << venta.getCantidad()
        << "$" << std::setw(13) << std::fixed <<
std::setprecision(2) << venta.getPrecio()
        << "$" << std::setw(13) << std::fixed <<
std::setprecision(2) << venta.calcularTotal()
        << std::endl;
}
}

void mostrarEstadisticas(const std::vector<Venta> &ventas) {
    if (ventas.empty()) {
        std::cout << "No hay ventas registradas." << std::endl;
        return;
    }

    double totalVentas = 0.0;
    int totalProductos = 0;
    for (const auto &venta : ventas) {
        totalVentas += venta.calcularTotal();
        totalProductos += venta.getCantidad();
    }
    double promedio = totalVentas / ventas.size();

    std::cout << "=== Estadísticas de Ventas ===" << std::endl;
    std::cout << "Total de transacciones: " << ventas.size() <<
std::endl;
    std::cout << "Total de productos vendidos: " << totalProductos <<
std::endl;
    std::cout << "Monto total vendido: $" << std::fixed <<
std::setprecision(2) << totalVentas << std::endl;
```



```
        std::cout << "Venta promedio: $" << std::fixed <<
std::setprecision(2) << promedio << std::endl;
    }

int main() {
    int numeroFactura = 1;
    std::vector<Venta> ventas = cargarVentas(numeroFactura);

    int opcion;
    do {
        mostrarMenu();
        opcion = leerEntero("Seleccione una opción: ");
        switch (opcion) {
            case 1: {
                std::string producto = leerTextoNoVacio("Nombre del
producto: ");
                int cantidad = leerEntero("Cantidad: ");
                double precio = leerDoublePositivo("Precio unitario: $");
                Venta venta(numeroFactura, producto, cantidad, precio);
                ventas.push_back(venta);
                numeroFactura++;
                guardarVentas(ventas);
                std::cout << "Venta registrada. Total: $" << std::fixed
<< std::setprecision(2)
                << venta.calcularTotal() << std::endl;
                break;
            }
            case 2: {
                if (ventas.empty()) {
                    std::cout << "No hay ventas registradas." <<
std::endl;
                    break;
                }
                int numero = leerEntero("Número de factura: ");
                Venta *venta = buscarVentaPorNumero(ventas, numero);
                if (venta == nullptr) {
                    std::cout << "Factura no encontrada." << std::endl;
                } else {
                    std::string factura = venta->generarFactura();
                    guardarFactura(factura);
                    std::cout << factura << std::endl;
                    std::cout << "Factura guardada en " <<
ARCHIVO_FACTURAS << std::endl;
                }
                break;
            }
            case 3:
                mostrarHistorialVentas(ventas);
        }
    }
```



```
        break;
    case 4:
        mostrarEstadisticas(ventas);
        break;
    case 5:
        std::cout << "Gracias por usar el Sistema de Ventas.
❖Hasta luego!" << std::endl;
        break;
    default:
        std::cout << "Opci❖n inv❖lida. Intente de nuevo." <<
std::endl;
    }
    std::cout << std::endl;
} while (opcion != 5);

guardarVentas(ventas);
return 0;
}
```

Casos de pruebas

```
=== Sistema de Ventas ===
1. Registrar venta
2. Generar factura
3. Mostrar historial de ventas
4. Ver estadísticas
5. Salir
Seleccione una opción: 1
Nombre del producto: Papas
Cantidad: 2
Precio unitario: $0.50
Venta registrada. Total: $1.00

=== Sistema de Ventas ===
1. Registrar venta
2. Generar factura
3. Mostrar historial de ventas
4. Ver estadísticas
5. Salir
Seleccione una opción: 4
```

Ejercicio 10 – Mini Proyecto Integrador

Objetivo: Desarrollar un sistema completo aplicando POO y archivos.

- Requerimientos:
 - Implementar CRUD.
 - Persistencia en archivos.
 - Menú interactivo y validaciones.
- Aplicar:
 - POO



- Archivos
- Métodos
- Validaciones

Análisis

Vamos a crear un programa muchísimo más complejo porque vamos a utilizar todos los temas aprendidos aplicados a un proyecto pequeño.

Código en Java

```
import java.io.BufferedReader;
import java.io.BufferedWriter;
import java.io.File;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;
import java.util.Scanner;

public class Ejercicio10 {

    private static final String ARCHIVO_EMPLEADOS = "empleados.txt";
    private static List<Empleado> empleados = new ArrayList<>();
    private static int ultimoId = 0;

    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        cargarEmpleados();

        int opcion;
        do {
            mostrarMenuPrincipal();
            opcion = leerEntero(scanner, "Seleccione una opción: ");
            switch (opcion) {
                case 1:
                    agregarEmpleado(scanner);
                    break;
                case 2:
                    listarTodosEmpleados();
                    break;
                case 3:
                    buscarEmpleado(scanner);
                    break;
                case 4:
                    actualizarEmpleado(scanner);
                    break;
                case 5:
                    break;
            }
        } while (opcion != 5);
    }
}
```



```
        eliminarEmpleado(scanner);
        break;
    case 6:
        mostrarEstadisticas();
        break;
    case 7:
        System.out.println("Gracias por usar el Sistema.
¡Hasta luego!");
        break;
    default:
        System.out.println("Opción inválida. Intente de
nuevo.");
    }
    System.out.println();
} while (opcion != 7);

guardarEmpleados();
scanner.close();
}

private static void mostrarMenuPrincipal() {
    System.out.println("=====
");
    System.out.println("||      SISTEMA DE GESTIÓN DE
EMPLEADOS      ||");
    System.out.println("=====
");
    System.out.println("1. Agregar empleado (CREATE)");
    System.out.println("2. Listar empleados (READ)");
    System.out.println("3. Buscar empleado");
    System.out.println("4. Actualizar empleado (UPDATE)");
    System.out.println("5. Eliminar empleado (DELETE)");
    System.out.println("6. Ver estadísticas");
    System.out.println("7. Salir");
}

private static int leerEntero(Scanner scanner, String mensaje) {
    while (true) {
        System.out.print(mensaje);
        String linea = scanner.nextLine().trim();
        try {
            return Integer.parseInt(linea);
        } catch (NumberFormatException e) {
            System.out.println("Debe ingresar un número válido.");
        }
    }
}
```



```
private static double leerDoublePositivo(Scanner scanner, String
mensaje) {
    while (true) {
        System.out.print(mensaje);
        String linea = scanner.nextLine().trim();
        try {
            double valor = Double.parseDouble(linea);
            if (valor < 0) {
                System.out.println("Debe ingresar un valor no
negativo.");
            } else {
                return valor;
            }
        } catch (NumberFormatException e) {
            System.out.println("Debe ingresar un número válido.");
        }
    }
}

private static String leerTextoNoVacio(Scanner scanner, String
mensaje) {
    while (true) {
        System.out.print(mensaje);
        String texto = scanner.nextLine().trim();
        if (!texto.isEmpty()) {
            return texto;
        }
        System.out.println("El texto no puede estar vacío.");
    }
}

private static String leerEmail(Scanner scanner, String mensaje) {
    while (true) {
        String email = leerTextoNoVacio(scanner, mensaje);
        if (email.contains("@") && email.contains(".")) {
            return email;
        }
        System.out.println("Email inválido. Debe contener @ y un
dominio.");
    }
}

private static String leerTelefono(Scanner scanner, String mensaje) {
    while (true) {
        String telefono = leerTextoNoVacio(scanner, mensaje);
        if (telefono.matches("\\d{7,}")) {
            return telefono;
        }
    }
}
```




```
        System.out.println("Teléfono inválido. Debe tener al menos 7
dígitos.");
    }
}

private static void agregarEmpleado(Scanner scanner) {
    System.out.println("\n--- Agregar Empleado ---");
    String nombre = leerTextoNoVacio(scanner, "Nombre completo: ");
    String puesto = leerTextoNoVacio(scanner, "Puesto: ");
    String email = leerEmail(scanner, "Email: ");
    String telefono = leerTelefono(scanner, "Teléfono: ");
    double salario = leerDoublePositivo(scanner, "Salario: $");
    String departamento = leerTextoNoVacio(scanner, "Departamento:
");

    ultimoId++;
    Empleado empleado = new Empleado(ultimoId, nombre, puesto, email,
telefono, salario, departamento);
    empleados.add(empleado);
    guardarEmpleados();
    System.out.println("✓ Empleado agregado con ID: " + ultimoId);
}

private static void listarTodosEmpleados() {
    if (empleados.isEmpty()) {
        System.out.println("No hay empleados registrados.");
        return;
    }

    System.out.println("\n=== Lista de Empleados ===");
    System.out.printf("%-4s %-20s %-15s %-20s %-15s %-12s %-15s\n",
        "ID", "Nombre", "Puesto", "Email", "Teléfono", "Salario",
"Depto.");
    System.out.println("-".repeat(110));

    for (Empleado emp : empleados) {
        System.out.printf("%-4d %-20s %-15s %-20s %-15s $%-11.2f %-
15s\n",
            emp.getId(),
            truncar(emp.getNombre(), 18),
            truncar(emp.getPuesto(), 13),
            truncar(emp.getEmail(), 18),
            truncar(emp.getTelefono(), 13),
            emp.getSalario(),
            truncar(emp.getDepartamento(), 13));
    }
}
```



```
private static void buscarEmpleado(Scanner scanner) {
    int id = leerEntero(scanner, "Ingrese el ID del empleado a
buscar: ");
    Empleado empleado = buscarPorId(id);

    if (empleado != null) {
        System.out.println("\n=== Información del Empleado ===");
        System.out.println(empleado);
    } else {
        System.out.println("Empleado no encontrado.");
    }
}

private static Empleado buscarPorId(int id) {
    for (Empleado emp : empleados) {
        if (emp.getId() == id) {
            return emp;
        }
    }
    return null;
}

private static void actualizarEmpleado(Scanner scanner) {
    System.out.println("\n--- Actualizar Empleado ---");
    int id = leerEntero(scanner, "Ingrese el ID del empleado a
actualizar: ");
    Empleado empleado = buscarPorId(id);

    if (empleado == null) {
        System.out.println("Empleado no encontrado.");
        return;
    }

    System.out.println("Datos actuales:");
    System.out.println(empleado);
    System.out.println("\nIngrese los nuevos datos (o presione Enter
para mantener el actual):");

    String nombre = scanner.nextLine().trim();
    if (!nombre.isEmpty()) {
        empleado.setNombre(nombre);
    }

    String puesto = scanner.nextLine().trim();
    if (!puesto.isEmpty()) {
        empleado.setPuesto(puesto);
    }
}
```



```
String email = scanner.nextLine().trim();
if (!email.isEmpty() && email.contains("@")) {
    empleado.setEmail(email);
}

String telefono = scanner.nextLine().trim();
if (!telefono.isEmpty() && telefono.matches("\\d{7,}")) {
    empleado.setTelefono(telefono);
}

System.out.print("Nuevo salario: $");
String salarioStr = scanner.nextLine().trim();
if (!salarioStr.isEmpty()) {
    try {
        double salario = Double.parseDouble(salarioStr);
        if (salario >= 0) {
            empleado.setSalario(salario);
        }
    } catch (NumberFormatException e) {
        // Mantener salario anterior
    }
}

String departamento = scanner.nextLine().trim();
if (!departamento.isEmpty()) {
    empleado.setDepartamento(departamento);
}

guardarEmpleados();
System.out.println("✓ Empleado actualizado exitosamente.");
}

private static void eliminarEmpleado(Scanner scanner) {
    int id = leerEntero(scanner, "Ingrese el ID del empleado a
eliminar: ");
    Empleado empleado = buscarPorId(id);

    if (empleado == null) {
        System.out.println("Empleado no encontrado.");
        return;
    }

    System.out.println("¿Desea eliminar a: " + empleado.getNombre() +
"? (s/n): ");
    String confirmacion = scanner.nextLine().trim().toLowerCase();

    if (confirmacion.equals("s")) {
        empleados.remove(empleado);
    }
}
```



```
        guardarEmpleados();
        System.out.println("✓ Empleado eliminado exitosamente.");
    } else {
        System.out.println("Operación cancelada.");
    }
}

private static void mostrarEstadisticas() {
    if (empleados.isEmpty()) {
        System.out.println("No hay empleados registrados.");
        return;
    }

    double totalSalarios = 0;
    double salarioMaximo = 0;
    double salarioMinimo = Double.MAX_VALUE;
    String empleadoMayorSalario = "";
    String empleadoMenorSalario = "";

    for (Empleado emp : empleados) {
        totalSalarios += emp.getSalario();
        if (emp.getSalario() > salarioMaximo) {
            salarioMaximo = emp.getSalario();
            empleadoMayorSalario = emp.getNombre();
        }
        if (emp.getSalario() < salarioMinimo) {
            salarioMinimo = emp.getSalario();
            empleadoMenorSalario = emp.getNombre();
        }
    }

    double promedio = totalSalarios / empleados.size();

    System.out.println("\n=== Estadísticas de Nómina ===");
    System.out.printf("Total de empleados: %d\n", empleados.size());
    System.out.printf("Nómina total: $%.2f\n", totalSalarios);
    System.out.printf("Salario promedio: $%.2f\n", promedio);
    System.out.printf("Mayor salario: $%.2f (%s)\n", salarioMaximo,
empleadoMayorSalario);
    System.out.printf("Menor salario: $%.2f (%s)\n", salarioMinimo,
empleadoMenorSalario);
}

private static String truncar(String texto, int longitud) {
    if (texto.length() <= longitud) {
        return texto;
    }
    return texto.substring(0, longitud - 2) + "..";
}
```



```
}

private static void cargarEmpleados() {
    File archivo = new File(ARCHIVO_EMPLEADOS);
    if (!archivo.exists()) {
        return;
    }

    try (BufferedReader reader = new BufferedReader(new
FileReader(archivo))) {
        String linea;
        while ((linea = reader.readLine()) != null) {
            String[] partes = linea.split("\\|");
            if (partes.length == 7) {
                int id = Integer.parseInt(partes[0]);
                String nombre = partes[1];
                String puesto = partes[2];
                String email = partes[3];
                String telefono = partes[4];
                double salario = Double.parseDouble(partes[5]);
                String departamento = partes[6];

                Empleado empleado = new Empleado(id, nombre, puesto,
email, telefono, salario, departamento);
                empleados.add(empleado);

                if (id > ultimoId) {
                    ultimoId = id;
                }
            }
        }
    } catch (IOException | NumberFormatException e) {
        System.out.println("Error al cargar empleados: " +
e.getMessage());
    }
}

private static void guardarEmpleados() {
    try (BufferedWriter writer = new BufferedWriter(new
FileWriter(ARCHIVO_EMPLEADOS))) {
        for (Empleado empleado : empleados) {
            writer.write(empleado.toRegistro());
            writer.newLine();
        }
    } catch (IOException e) {
        System.out.println("Error al guardar empleados: " +
e.getMessage());
    }
}
```



```
}  
}  
  
class Empleado {  
  
    private final int id;  
    private String nombre;  
    private String puesto;  
    private String email;  
    private String telefono;  
    private double salario;  
    private String departamento;  
  
    public Empleado(int id, String nombre, String puesto, String email,  
String telefono, double salario, String departamento) {  
        this.id = id;  
        this.nombre = nombre;  
        this.puesto = puesto;  
        this.email = email;  
        this.telefono = telefono;  
        this.salario = salario;  
        this.departamento = departamento;  
    }  
  
    public int getId() {  
        return id;  
    }  
  
    public String getNombre() {  
        return nombre;  
    }  
  
    public void setNombre(String nombre) {  
        this.nombre = nombre;  
    }  
  
    public String getPuesto() {  
        return puesto;  
    }  
  
    public void setPuesto(String puesto) {  
        this.puesto = puesto;  
    }  
  
    public String getEmail() {  
        return email;  
    }  
}
```



```
public void setEmail(String email) {
    this.email = email;
}

public String getTelefono() {
    return telefono;
}

public void setTelefono(String telefono) {
    this.telefono = telefono;
}

public double getSalario() {
    return salario;
}

public void setSalario(double salario) {
    this.salario = salario;
}

public String getDepartamento() {
    return departamento;
}

public void setDepartamento(String departamento) {
    this.departamento = departamento;
}

public String toRegistro() {
    return id + "|" + nombre + "|" + puesto + "|" + email + "|" +
telefono + "|" + salario + "|" + departamento;
}

@Override
public String toString() {
    return String.format(
        "ID: %d%n" +
        "Nombre: %s%n" +
        "Puesto: %s%n" +
        "Email: %s%n" +
        "Teléfono: %s%n" +
        "Salario: $%.2f%n" +
        "Departamento: %s",
        id, nombre, puesto, email, telefono, salario,
departamento);
}
}
```

Código en C++



```
#include <fstream>
#include <iostream>
#include <iomanip>
#include <sstream>
#include <string>
#include <vector>
#include <limits>
#include <regex>
#include <algorithm>

const std::string ARCHIVO_EMPLEADOS = "empleados.txt";

class Empleado {
public:
    Empleado(int id, const std::string &nombre, const std::string
&puesto,
            const std::string &email, const std::string &telefono,
            double salario, const std::string &departamento)
        : id(id), nombre(nombre), puesto(puesto), email(email),
telefono(telefono),
        salario(salario), departamento(departamento) {}

    int getId() const { return id; }
    const std::string &getNombre() const { return nombre; }
    const std::string &getPuesto() const { return puesto; }
    const std::string &getEmail() const { return email; }
    const std::string &getTelefono() const { return telefono; }
    double getSalario() const { return salario; }
    const std::string &getDepartamento() const { return departamento; }

    void setNombre(const std::string &value) { nombre = value; }
    void setPuesto(const std::string &value) { puesto = value; }
    void setEmail(const std::string &value) { email = value; }
    void setTelefono(const std::string &value) { telefono = value; }
    void setSalario(double value) { salario = value; }
    void setDepartamento(const std::string &value) { departamento =
value; }

    std::string toRegistro() const {
        std::ostringstream out;
        out << id << "|" << nombre << "|" << puesto << "|" << email <<
"|"
            << telefono << "|" << salario << "|" << departamento;
        return out.str();
    }

    std::string toString() const {
        std::ostringstream out;
```




```
        out << "ID: " << id << "\n"
        << "Nombre: " << nombre << "\n"
        << "Puesto: " << puesto << "\n"
        << "Email: " << email << "\n"
        << "Teléfono: " << telefono << "\n"
        << "Salario: $" << std::fixed << std::setprecision(2) <<
salario << "\n"
        << "Departamento: " << departamento;
        return out.str();
    }

private:
    int id;
    std::string nombre;
    std::string puesto;
    std::string email;
    std::string telefono;
    double salario;
    std::string departamento;
};

int leerEntero(const std::string &mensaje) {
    while (true) {
        std::cout << mensaje;
        int valor;
        if (std::cin >> valor) {
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
            return valor;
        }
        std::cout << "Debe ingresar un número válido." << std::endl;
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
    }
}

double leerDoubleNoNegativo(const std::string &mensaje) {
    while (true) {
        std::cout << mensaje;
        double valor;
        if (std::cin >> valor) {
            std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
            if (valor >= 0) {
                return valor;
            }
        }
    }
}
```



```
        std::cout << "Debe ingresar un valor no negativo." <<
std::endl;
    } else {
        std::cout << "Debe ingresar un numero válido." << std::endl;
        std::cin.clear();
        std::cin.ignore(std::numeric_limits<std::streamsize>::max(),
'\n');
    }
}
}

std::string leerTextoNoVacio(const std::string &mensaje) {
    while (true) {
        std::cout << mensaje;
        std::string texto;
        std::getline(std::cin, texto);
        if (!texto.empty()) {
            return texto;
        }
        std::cout << "El texto no puede estar vacío." << std::endl;
    }
}

std::string leerEmail(const std::string &mensaje) {
    std::regex patron(R"([^\s@]+@[^\s@]+\.[^\s@]+)");
    while (true) {
        std::string email = leerTextoNoVacio(mensaje);
        if (std::regex_match(email, patron)) {
            return email;
        }
        std::cout << "Email inválido. Debe contener @ y un dominio." <<
std::endl;
    }
}

std::string leerTelefono(const std::string &mensaje) {
    std::regex patron(R"(\d{7,})");
    while (true) {
        std::string telefono = leerTextoNoVacio(mensaje);
        if (std::regex_match(telefono, patron)) {
            return telefono;
        }
        std::cout << "Teléfono inválido. Debe tener al menos 7 dígitos."
<< std::endl;
    }
}

std::vector<Empleado> cargarEmpleados(int &ultimoId) {
```



```
std::vector<Empleado> empleados;
std::ifstream archivo(ARCHIVO_EMPLEADOS);
if (!archivo.is_open()) {
    return empleados;
}

std::string linea;
while (std::getline(archivo, linea)) {
    std::istringstream ss(linea);
    std::string idStr, nombre, puesto, email, telefono, salarioStr,
departamento;
    if (std::getline(ss, idStr, '|') && std::getline(ss, nombre, '|')
&&
        std::getline(ss, puesto, '|') && std::getline(ss, email, '|')
&&
        std::getline(ss, telefono, '|') && std::getline(ss,
salarioStr, '|') &&
        std::getline(ss, departamento)) {
        try {
            int id = std::stoi(idStr);
            double salario = std::stod(salarioStr);
            empleados.emplace_back(id, nombre, puesto, email,
telefono, salario, departamento);
            if (id > ultimoId) {
                ultimoId = id;
            }
        } catch (...) {
            // ignorar línea malformada
        }
    }
}

return empleados;
}

void guardarEmpleados(const std::vector<Empleado> &empleados) {
    std::ofstream archivo(ARCHIVO_EMPLEADOS);
    if (!archivo.is_open()) {
        std::cout << "Error al guardar empleados." << std::endl;
        return;
    }

    for (const auto &empleado : empleados) {
        archivo << empleado.toRegistro() << "\n";
    }
}

Empleado *buscarPorId(std::vector<Empleado> &empleados, int id) {
```



```
for (auto &empleado : empleados) {
    if (empleado.getId() == id) {
        return &empleado;
    }
}
return nullptr;
}

std::string truncar(const std::string &texto, size_t longitud) {
    if (texto.size() <= longitud) {
        return texto;
    }
    return texto.substr(0, longitud - 2) + "..";
}

void mostrarMenuPrincipal() {
    std::cout << "┌───────────────────────────────────────────┐" <<
std::endl;
    std::cout << "│          SISTEMA DE GESTIÓN DE EMPLEADOS          │" <<
std::endl;
    std::cout << "└───────────────────────────────────────────┘" <<
std::endl;
    std::cout << "1. Agregar empleado " << std::endl;
    std::cout << "2. Listar empleados" << std::endl;
    std::cout << "3. Buscar empleado" << std::endl;
    std::cout << "4. Actualizar empleado" << std::endl;
    std::cout << "5. Eliminar empleado" << std::endl;
    std::cout << "6. Ver estadísticas" << std::endl;
    std::cout << "7. Salir" << std::endl;
}

void listarTodosEmpleados(const std::vector<Empleado> &empleados) {
    if (empleados.empty()) {
        std::cout << "No hay empleados registrados." << std::endl;
        return;
    }

    std::cout << "\n=== Lista de Empleados ===" << std::endl;
    std::cout << std::left << std::setw(4) << "ID" << std::setw(20) <<
"Nombre" << std::setw(15)
        << "Puesto" << std::setw(20) << "Email" << std::setw(15) <<
"Teléfono" << std::setw(12)
        << "Salario" << std::setw(15) << "Depto." << std::endl;
    std::cout << std::string(110, '-') << std::endl;

    for (const auto &emp : empleados) {
        std::cout << std::left << std::setw(4) << emp.getId()
            << std::setw(20) << truncar(emp.getNombre(), 18)
```



```
<< std::setw(15) << truncar(emp.getPuesto(), 13)
<< std::setw(20) << truncar(emp.getEmail(), 18)
<< std::setw(15) << truncar(emp.getTelefono(), 13)
<< "$" << std::setw(11) << std::fixed <<
std::setprecision(2) << emp.getSalario()
<< std::setw(15) << truncar(emp.getDepartamento(), 13)
<< std::endl;
    }
}

int main() {
    int ultimoId = 0;
    std::vector<Empleado> empleados = cargarEmpleados(ultimoId);
    int opcion;

    do {
        mostrarMenuPrincipal();
        opcion = leerEntero("Seleccione una opción: ");
        switch (opcion) {
            case 1: {
                std::cout << "\n--- Agregar Empleado ---" << std::endl;
                std::string nombre = leerTextoNoVacio("Nombre completo:
");

                std::string puesto = leerTextoNoVacio("Puesto: ");
                std::string email = leerEmail("Email: ");
                std::string telefono = leerTelefono("Teléfono: ");
                double salario = leerDoubleNoNegativo("Salario: $");
                std::string departamento =
leerTextoNoVacio("Departamento: ");

                ultimoId++;
                empleados.emplace_back(ultimoId, nombre, puesto, email,
telefono, salario, departamento);
                guardarEmpleados(empleados);
                std::cout << "✓ Empleado agregado con ID: " << ultimoId
<< std::endl;
                break;
            }
            case 2:
                listarTodosEmpleados(empleados);
                break;
            case 3: {
                int id = leerEntero("Ingrese el ID del empleado a buscar:
");

                Empleado *empleado = buscarPorId(empleados, id);
                if (empleado != nullptr) {
                    std::cout << "\n=== Información del Empleado ===" <<
std::endl;
```



```
        std::cout << empleado->toString() << std::endl;
    } else {
        std::cout << "Empleado no encontrado." << std::endl;
    }
    break;
}
case 4: {
    std::cout << "\n--- Actualizar Empleado ---" <<
std::endl;
    int id = leerEntero("Ingrese el ID del empleado a
actualizar: ");
    Empleado *empleado = buscarPorId(empleados, id);
    if (empleado == nullptr) {
        std::cout << "Empleado no encontrado." << std::endl;
        break;
    }
    std::cout << "Datos actuales:" << std::endl;
    std::cout << empleado->toString() << std::endl;
    std::cout << "\nIngrese los nuevos datos (o presione
Enter para mantener el actual):" << std::endl;

    std::string nombre;
    std::getline(std::cin, nombre);
    if (!nombre.empty()) empleado->setNombre(nombre);

    std::string puesto;
    std::getline(std::cin, puesto);
    if (!puesto.empty()) empleado->setPuesto(puesto);

    std::string email;
    std::getline(std::cin, email);
    if (!email.empty() && std::regex_match(email,
std::regex(R"((([^\s]+)@([^\s]+)\.([^\s]+)))") {
        empleado->setEmail(email);
    }

    std::string telefono;
    std::getline(std::cin, telefono);
    if (!telefono.empty() && std::regex_match(telefono,
std::regex(R"(^d{7,}$)")) {
        empleado->setTelefono(telefono);
    }

    std::cout << "Nuevo salario: $";
    std::string salarioStr;
    std::getline(std::cin, salarioStr);
    if (!salarioStr.empty()) {
        try {
```



```
        double salario = std::stod(salarioStr);
        if (salario >= 0) empleado->setSalario(salario);
    } catch (...) {
    }
}

std::string departamento;
std::getline(std::cin, departamento);
if (!departamento.empty()) empleado-
>setDepartamento(departamento);

guardarEmpleados(empleados);
std::cout << "✓ Empleado actualizado exitosamente." <<
std::endl;

break;
}
case 5: {
    int id = leerEntero("Ingrese el ID del empleado a
eliminar: ");
    Empleado *empleado = buscarPorId(empleados, id);
    if (empleado == nullptr) {
        std::cout << "Empleado no encontrado." << std::endl;
        break;
    }
    std::cout << "¿Desea eliminar a: " << empleado-
>getNombre() << "? (s/n): ";
    std::string confirmacion;
    std::getline(std::cin, confirmacion);
    if (confirmacion == "s" || confirmacion == "S") {
        empleados.erase(std::remove_if(empleados.begin(),
empleados.end(),
            [&](const Empleado &e) { return e.getId() == id;
}), empleados.end());
        guardarEmpleados(empleados);
        std::cout << "✓ Empleado eliminado exitosamente." <<
std::endl;

    } else {
        std::cout << "Operación cancelada." << std::endl;
    }
    break;
}
case 6: {
    if (empleados.empty()) {
        std::cout << "No hay empleados registrados." <<
std::endl;

        break;
    }
    double totalSalarios = 0.0;
```



```
double salarioMaximo = 0.0;
double salarioMinimo =
std::numeric_limits<double>::max();
std::string empleadoMayorSalario;
std::string empleadoMenorSalario;
for (const auto &emp : empleados) {
    totalSalarios += emp.getSalario();
    if (emp.getSalario() > salarioMaximo) {
        salarioMaximo = emp.getSalario();
        empleadoMayorSalario = emp.getNombre();
    }
    if (emp.getSalario() < salarioMinimo) {
        salarioMinimo = emp.getSalario();
        empleadoMenorSalario = emp.getNombre();
    }
}
double promedio = totalSalarios / empleados.size();
std::cout << "\n=== Estadísticas de Nómina ===" <<
std::endl;
std::cout << "Total de empleados: " << empleados.size()
<< std::endl;
std::cout << "Nómina total: $" << std::fixed <<
std::setprecision(2) << totalSalarios << std::endl;
std::cout << "Salario promedio: $" << std::fixed <<
std::setprecision(2) << promedio << std::endl;
std::cout << "Mayor salario: $" << std::fixed <<
std::setprecision(2) << salarioMaximo << " (" << empleadoMayorSalario <<
")" << std::endl;
std::cout << "Menor salario: $" << std::fixed <<
std::setprecision(2) << salarioMinimo << " (" << empleadoMenorSalario <<
")" << std::endl;
break;
}
case 7:
std::cout << "Gracias por usar el Sistema. ¡Hasta luego!"
<< std::endl;
break;
default:
std::cout << "Opción invalida. Intente de nuevo." <<
std::endl;
break;
}
std::cout << std::endl;
} while (opcion != 7);

guardarEmpleados(empleados);
return 0;
}
```




Casos de pruebas

```
SISTEMA DE GESTIÓN DE EMPLEADOS

1. Agregar empleado (CREATE)
2. Listar empleados (READ)
3. Buscar empleado
4. Actualizar empleado (UPDATE)
5. Eliminar empleado (DELETE)
6. Ver estadísticas
7. Salir
Seleccione una opción: 1

--- Agregar Empleado ---
Nombre completo: Bryan
Puesto: 1
Email: nunes@gmail.com
Teléfono: 0987654321
Salario: $100
Departamento: Atencion al cliente
? Empleado agregado con ID: 1

SISTEMA DE GESTIÓN DE EMPLEADOS

1. Agregar empleado (CREATE)
2. Listar empleados (READ)
3. Buscar empleado
4. Actualizar empleado (UPDATE)
5. Eliminar empleado (DELETE)
6. Ver estadísticas
```

2.7 Resultados obtenidos

De acuerdo con el tipo de trabajo, se plasmarán los resultados alcanzados en la guía práctica una vez ejecutadas las actividades. Se pueden emplear figuras y tablas las cuales deben ser numeradas.

2.8 Habilidades blandas empleadas en la práctica

- ☐ Liderazgo
- ☒ Trabajo en equipo
- ☒ Comunicación asertiva
- ☐ La empatía
- ☐ Pensamiento crítico
- ☐ Flexibilidad
- ☐ La resolución de conflictos
- ☐ Adaptabilidad
- ☒ Responsabilidad

2.9 Conclusiones

Concluimos que estos problemas nos ayudan a comprender el uso de la programación en un habiente mas laboras y complejo para prepararnos en la vida laboral.



2.10 Recomendaciones

- Poner en practica los temas aprendidos en clases para entender su funcionamiento.
- Saber cómo funciona la lógica del programa para no perderse al realizar el programa.
- Saber donde y cuando usar esos nuevos temas aprendidos.
- Saber temas complejos de la programación para tener un programa mas limpio y ordenado.

2.11 Referencias bibliográficas

No se utilizaron fuentes de investigación

2.12 Anexos

Todo esta adjuntado en un repositorio en github.

<https://github.com/Kevin-Verdesoto34/APE-05-Creaci-n-Lectura-y-Escritura-de-Ficheros>